INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN:

AN AGILE, ITERATIVE APPROACH

SATZINGER | JACKSON | BURD

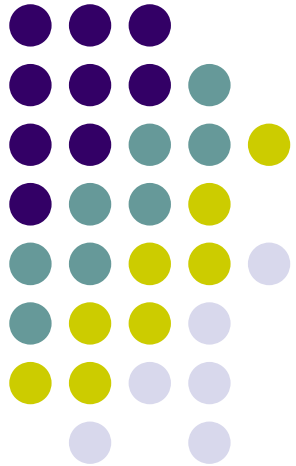
Chapter 10

Object-Oriented Design: Principles

Chapter 10

Introduction to Systems Analysis and Design: An Agile, Iteractive Approach 6th Ed

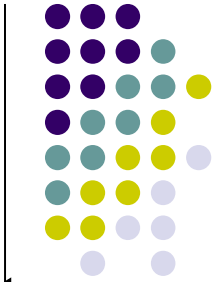
Satzinger, Jackson & Burd



Chapter 10 Outline

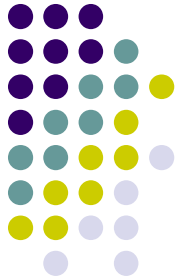
- Object-Oriented Design: Bridging from Analysis to Implementation （面向对象设计：分析与实施的桥梁）
- Object-Oriented Architectural Design （面向对象结构化设计）
- Fundamental Principles of Object-Oriented Detailed Design （面向对象细节设计的基本原则）
- Design Classes and the Design Class Diagram （类和设计类图）
- Detailed Design with CRC Cards （用 CRC 卡进行细节设计）
- Fundamental Detailed Design Principles （细节设计的基本原则）

Learning Objectives



- Explain the purpose and objectives of object-oriented design （面向对象设计的目的和目标）
- Develop UML component diagrams （开发 UML 组件图）
- Develop design class diagrams （开发设计类图）
- Use CRC cards to define class responsibilities and collaborations （使用 CRC 卡来定义类的职责和合作关联）
- Explain some of the fundamental principles of object-oriented design （面向对象设计的基本原则）

Overview

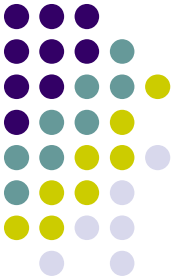


- This chapter and the next focus on designing software for the new system, at both the architectural （架构设计） and detailed level design （详细设计）
- For architectural design （架构设计）, the model is shown as a UML component diagram
- For detailed design （详细设计）, the main models are design class diagrams and sequence diagrams
- In this chapter, the CRC Cards technique is used to design the OO software

Chapter 10 Outline

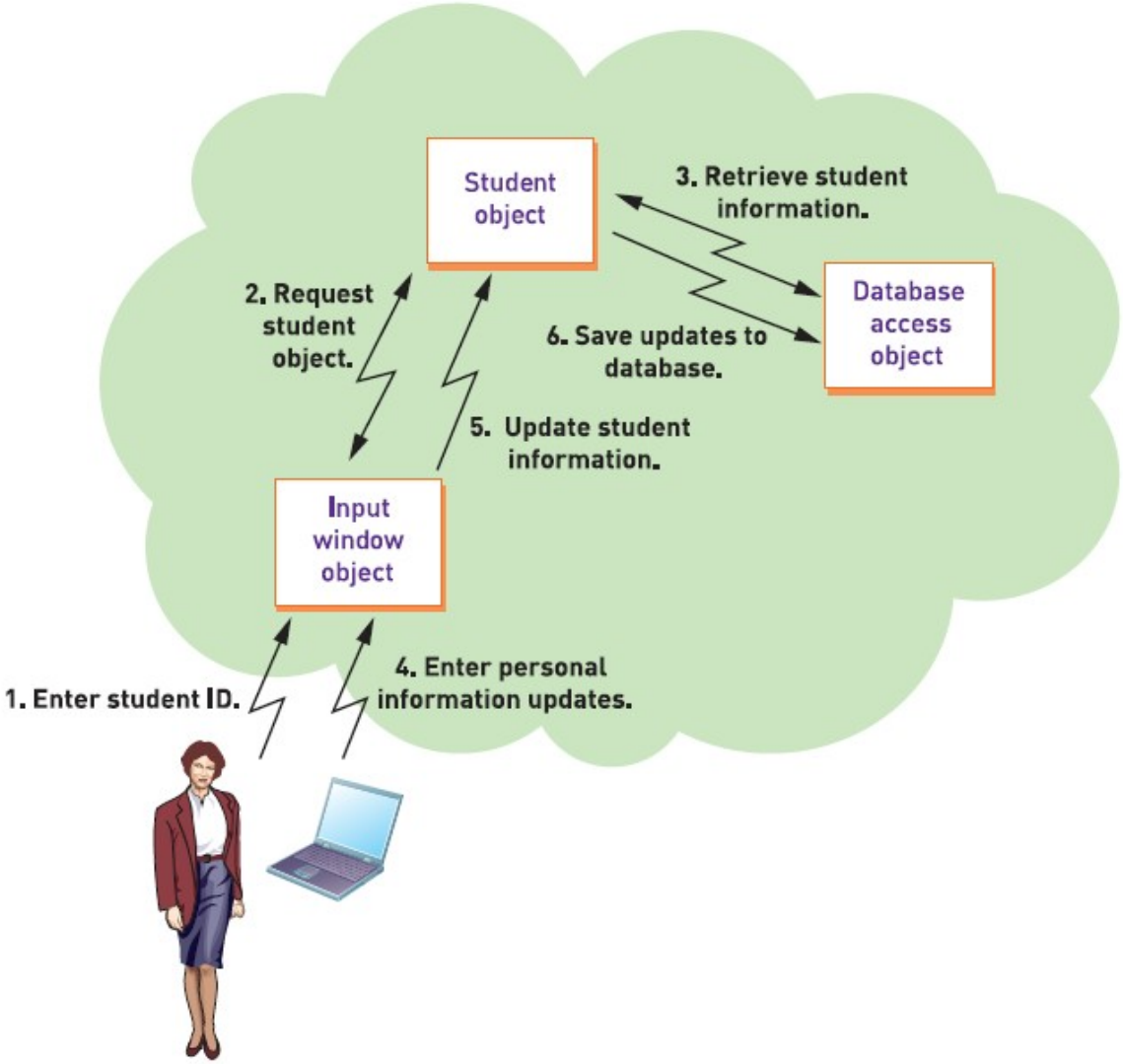
- Object-Oriented Design: Bridging from Analysis to Implementation （面向对象设计：分析与实施的桥梁)
- Object-Oriented Architectural Design
- Fundamental Principles of Object-Oriented Detailed Design
- Design Classes and the Design Class Diagram
- Detailed Design with CRC Cards
- Fundamental Detailed Design Principles

OO Design:
The Bridge from Analysis to Design



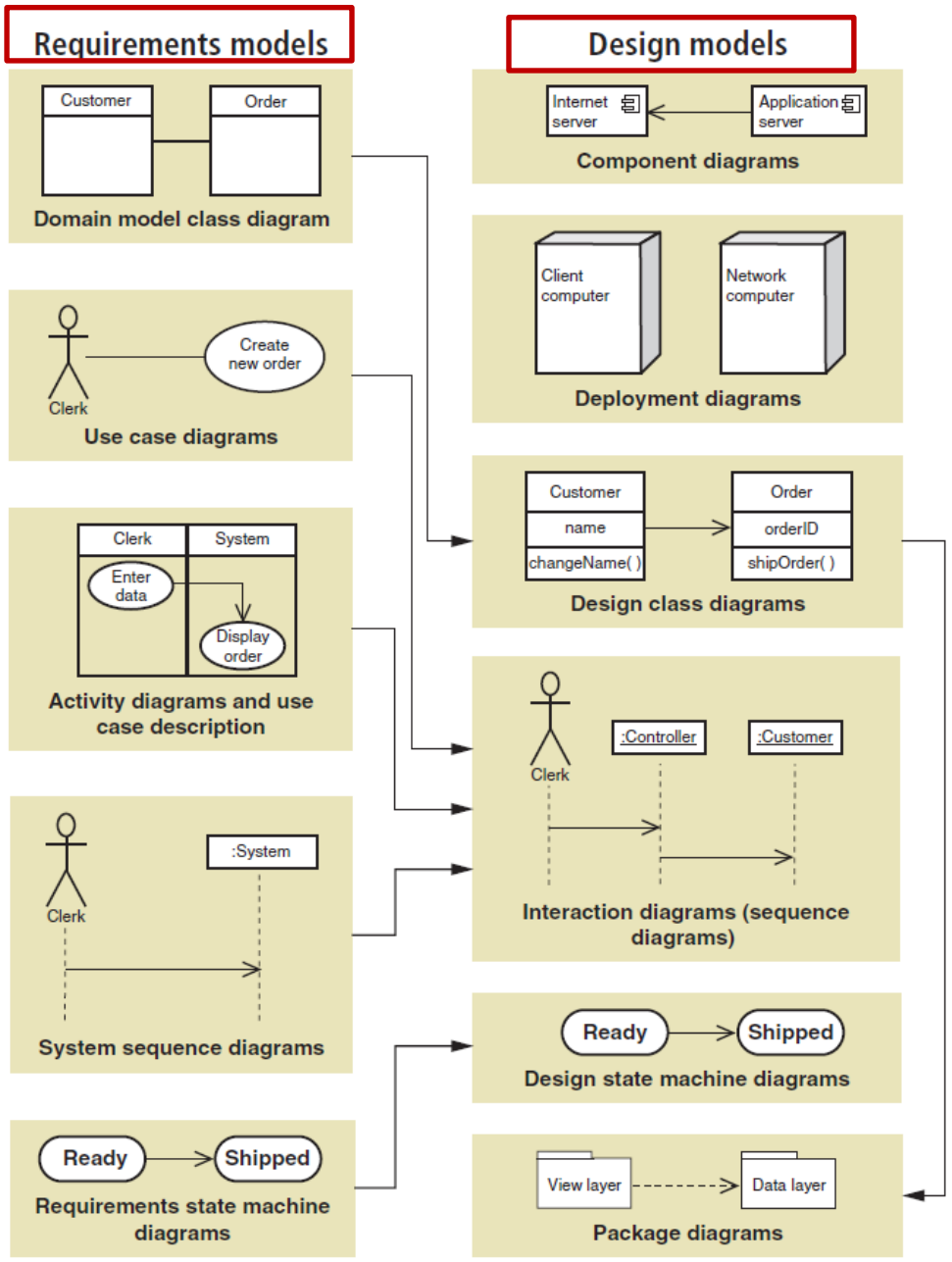
- OO Design: Process by which a set of detailed OO design models are built to be used for coding （ OO 设计：构建一组详细的 OO 设计模型以用编码的过程）
- Design models are created in parallel to actual coding/implementation with iterative SDLC （设计模型是通过迭代 SDLC 与实际编码 / 实现并行创建的）
- Agile approache says create models only if they are necessary. Simple detailed aspects don't need a design model before coding （敏捷方法认为，只有在必要时才创建模型）

Object-Oriented Program Flow
Three Layer Architecture



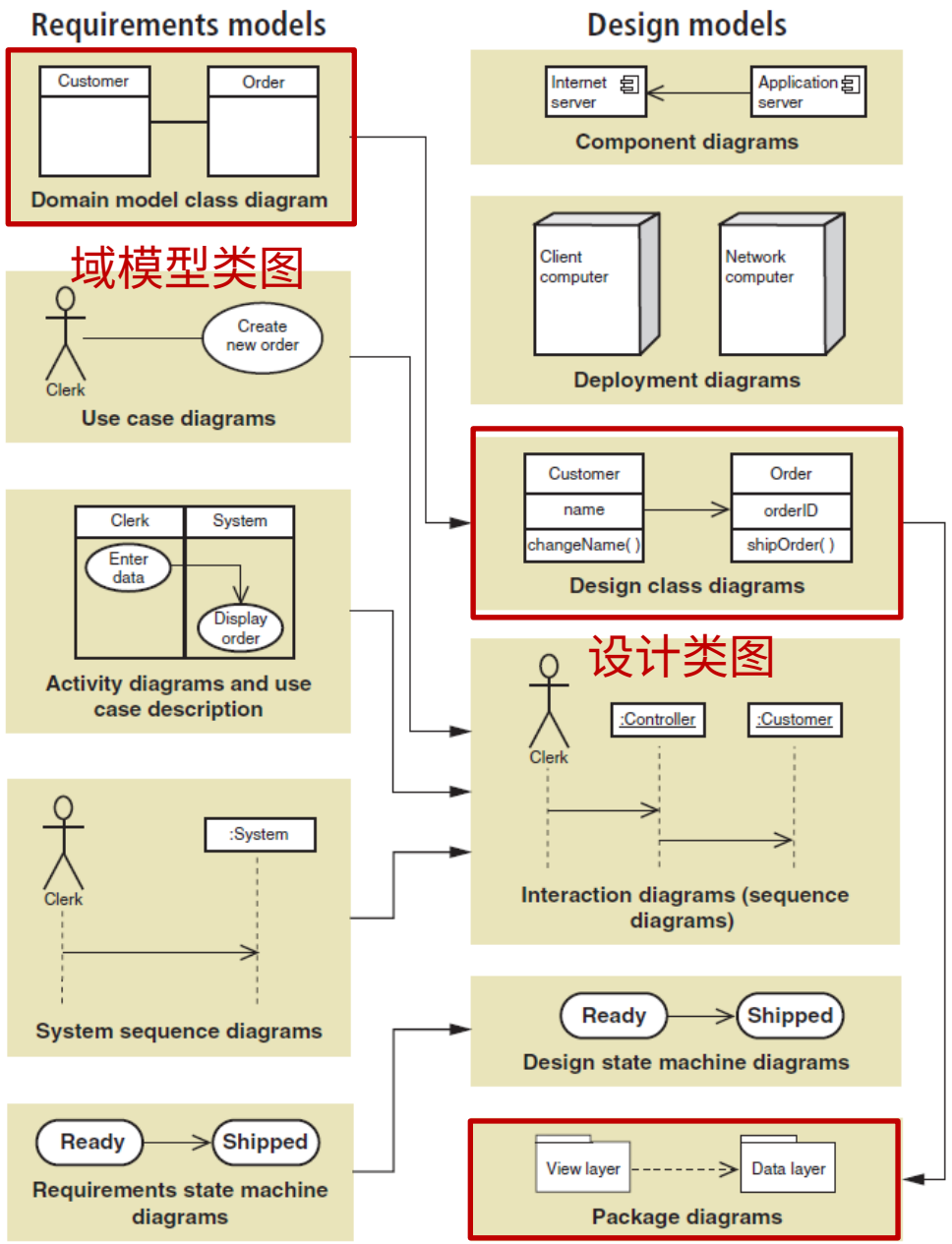
UML Requirements vs. Design Models

Diagrams are enhanced and extended



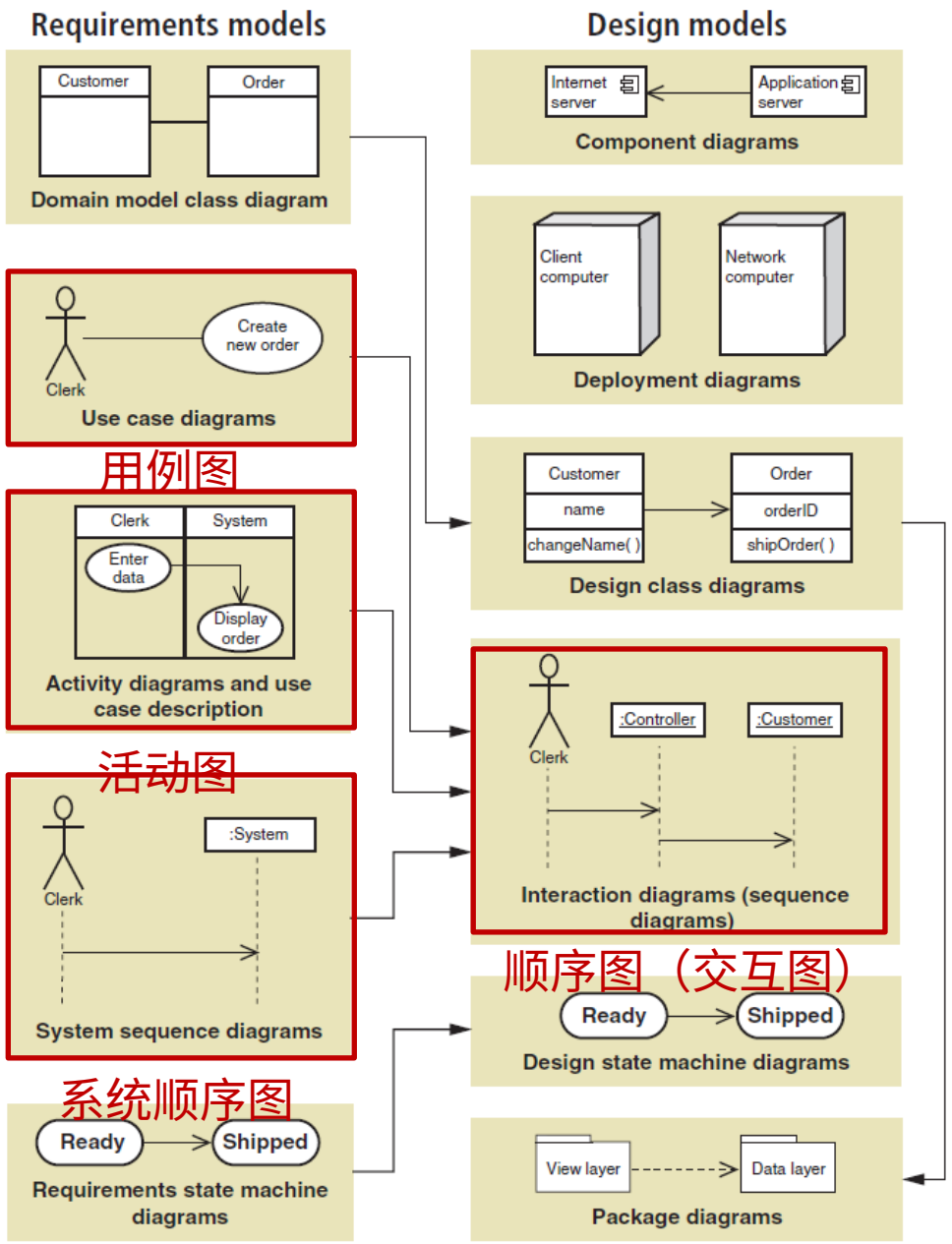
UML Requirements vs. Design Models

Diagrams are enhanced and extended



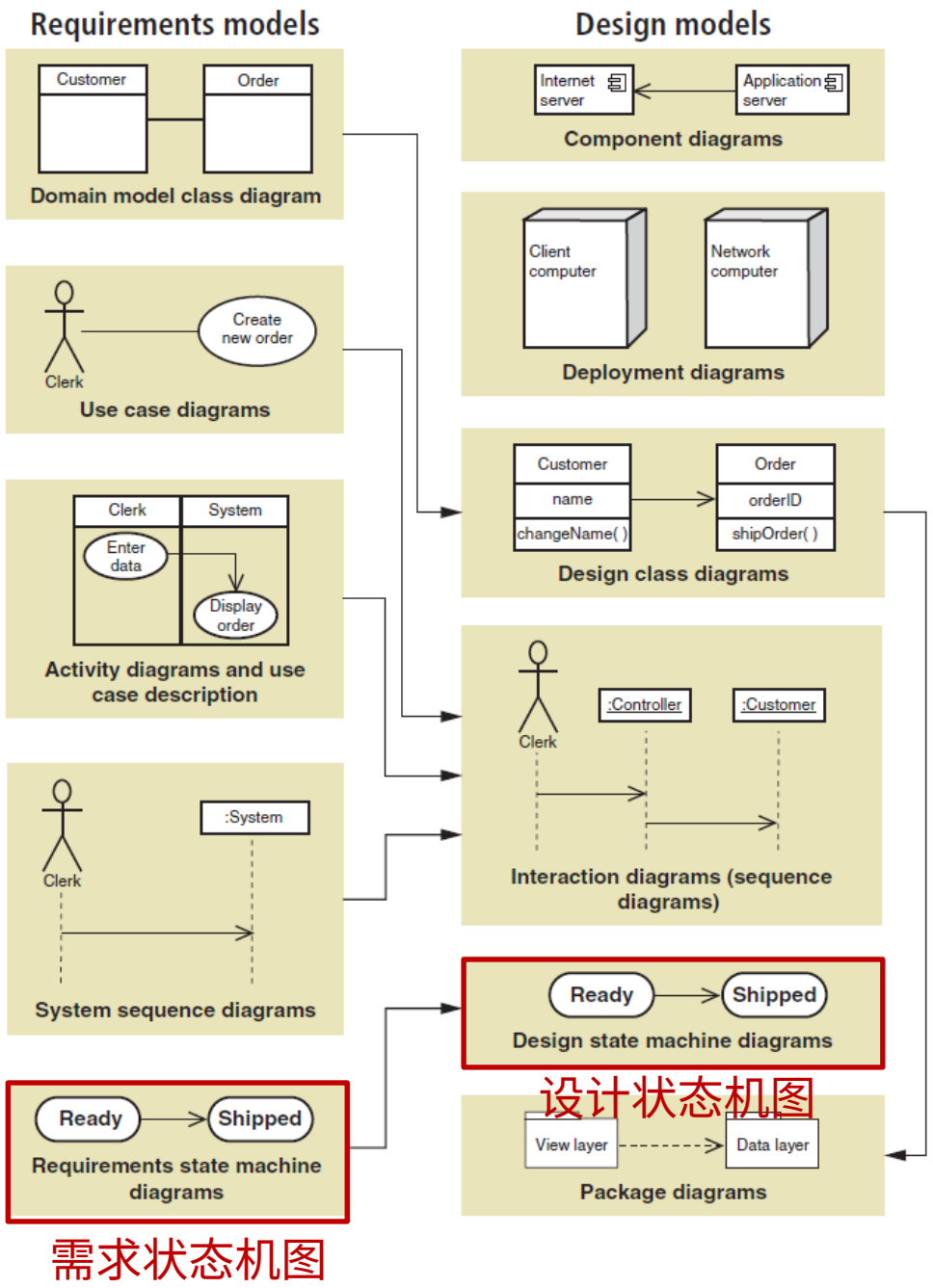
UML Requirements vs. Design Models

Diagrams are enhanced and extended

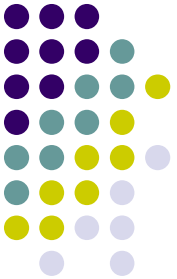


UML Requirements vs. Design Models

Diagrams are enhanced and extended



Chapter 10 Outline



- Object-Oriented Design: Bridging from Analysis to Implementation
- Object-Oriented Architectural Design （面向对象结构化设计）
- Fundamental Principles of Object-Oriented Detailed Design
- Design Classes and the Design Class Diagram
- Detailed Design with CRC Cards
- Fundamental Detailed Design Principles

Architectural Design

- 软件系统一般会分成两类：单用户系统和企业级系统
  - 企业级系统：在组织中的多个人员或组之间共享资源的系统
- 两种选择：基于 client-server 和 基于网络连接的

Design Issue	Client/Server Network System	Internet System
State	"Stateful" or state-based system—e.g., client/server connection is long term.	Stateless system—e.g., client/server connection is not long term and has no inherent memory.
Client configuration	Screens and forms that are programmed are displayed directly. Domain layer is often on the client or split between client and server machines.	Screens and forms are displayed only through a browser. They must conform to browser technology.
Server configuration	Application or data server directly connects to client tier.	Client tier connects indirectly to the application server through a Web server.

C-S 的连接是长期的

短期的、无长期记忆的

Architectural Design

- Enterprise-level system
  - a system that has shared resources among multiple people or groups in an organization
- Options are Client-Server or Internet Based
  - Each presents different issues

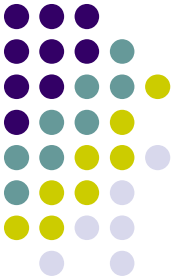
Design Issue	Client/Server Network System	Internet System
State	"Stateful" or state-based system—e.g., client/server connection is long term.	Stateless system—e.g., client/server connection is not long term and has no inherent memory.
Client configuration	Screens and forms that are programmed are displayed directly. Domain layer is often on the client or split between client and server machines. 直接显示	Screens and forms are displayed only through a browser. They must conform to browser technology. 通过浏览器显示
Server configuration	Application or data server directly connects to client tier.	Client tier connects indirectly to the application server through a Web server.

Architectural Design

- Enterprise-level system
  - a system that has shared resources among multiple people or groups in an organization
- Options are Client-Server or Internet Based
  - Each presents different issues

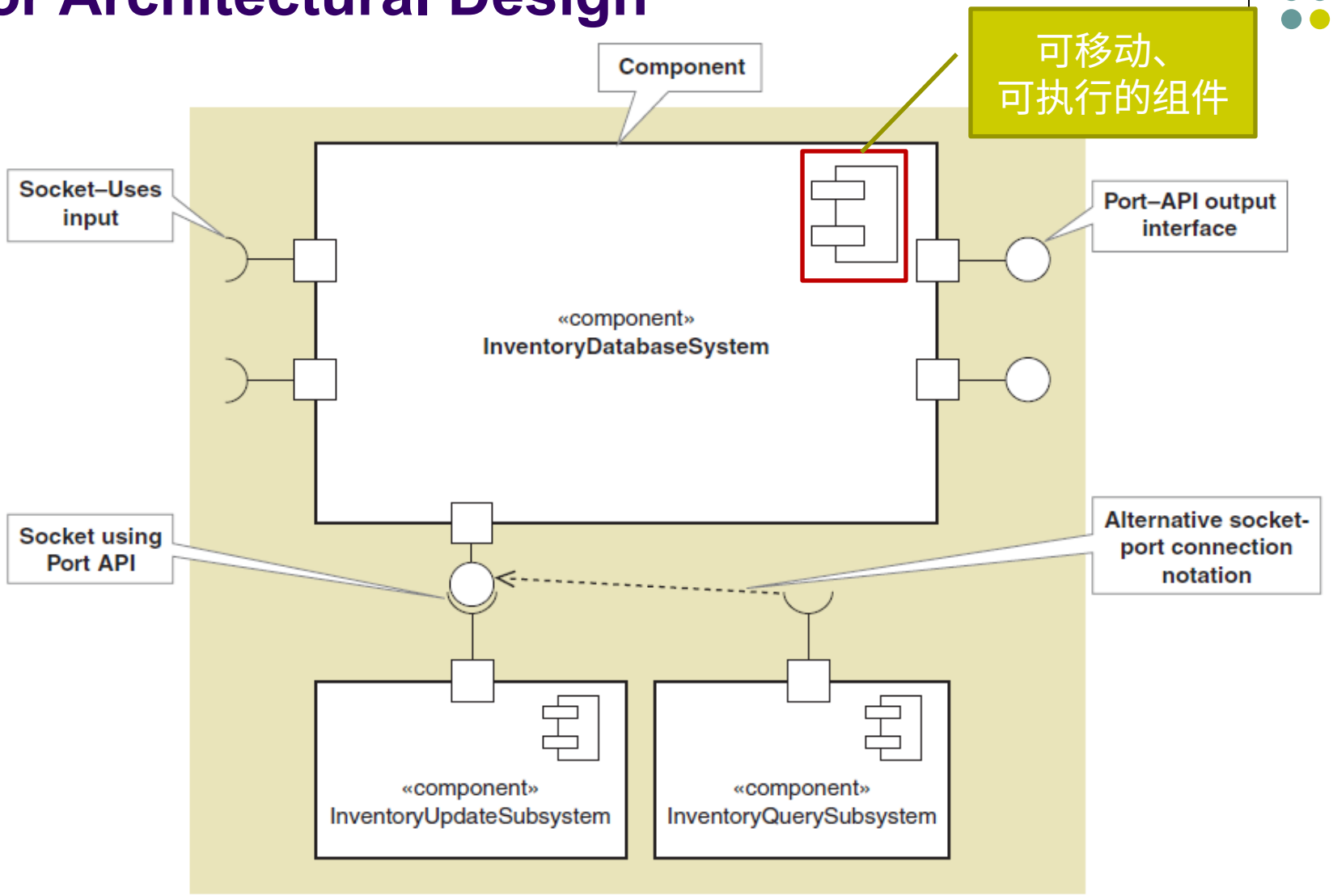
Design Issue	Client/Server Network System	Internet System
State	"Stateful" or state-based system—e.g., client/server connection is long term.	Stateless system—e.g., client/server connection is not long term and has no inherent memory.
Client configuration	Screens and forms that are programmed are displayed directly. Domain layer is often on the client or split between client and server machines.	Screens and forms are displayed only through a browser. They must conform to browser technology.
Server configuration	Application or data server directly connects to client tier. 直接地	Client tier connects indirectly to the application server through a Web server. 间接地

Component Diagram
for Architectural Design

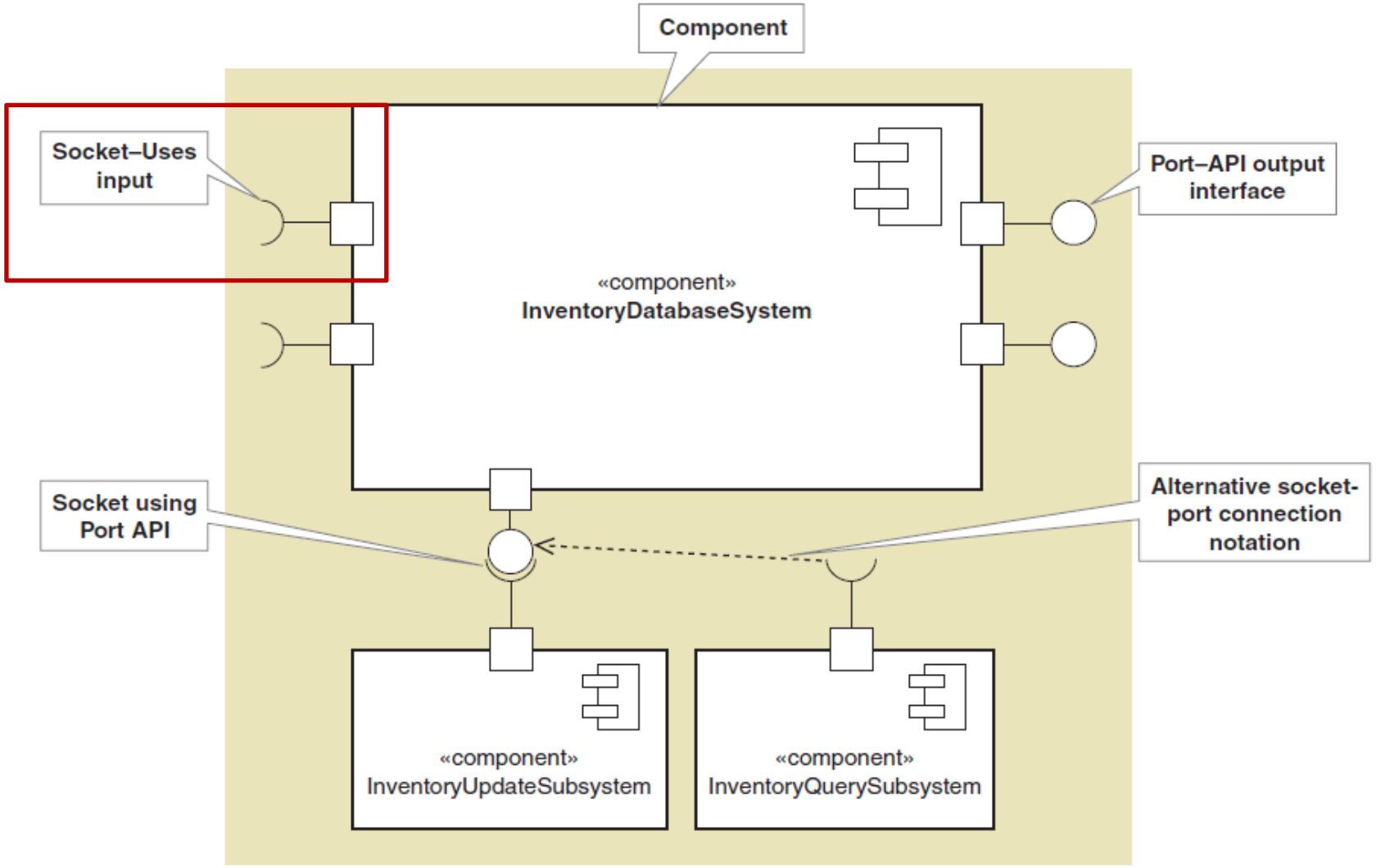


- Component diagram （组件图） –
  - A type of design diagram （设计图） that shows the overall system architecture （系统架构） and the logical components （逻辑组件） within it for how the system will be implemented
  - Identifies the logical, reusable, and transportable system components （系统组件） that define the system architecture
  - The essential element of a component diagram is the component element with its API （组件图的基本元素）.
- Application program interface (API) （应用程序接口）–
  - The set of public methods （公用方法） that are available to the outside world

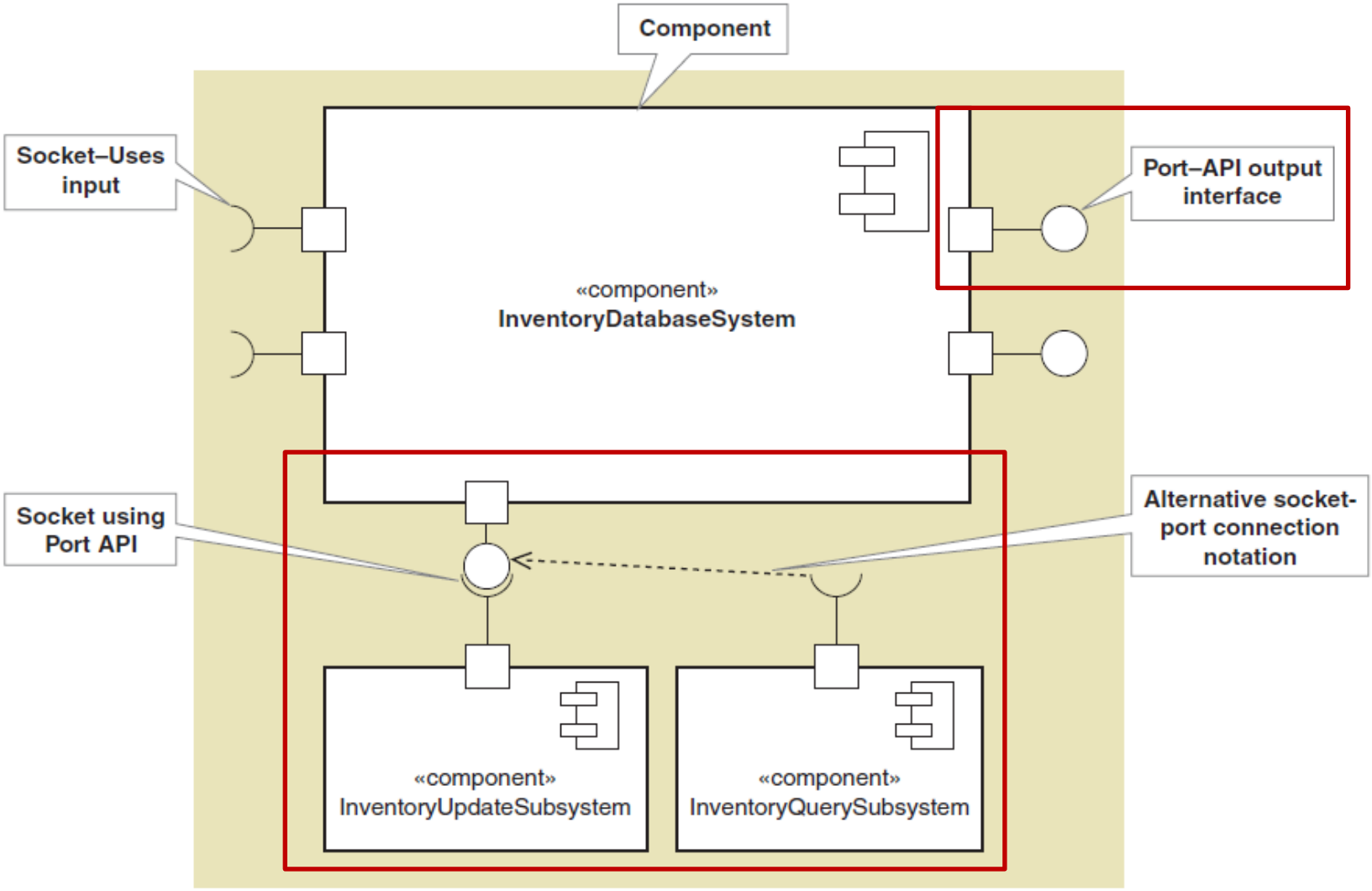
Component Diagram
for Architectural Design



Component Diagram
for Architectural Design



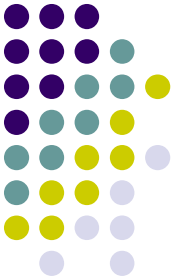
Component Diagram
for Architectural Design



Chapter 10 Outline

- Object-Oriented Design: Bridging from Analysis to Implementation
- Object-Oriented Architectural Design
- Fundamental Principles of Object-Oriented Detailed Design （面向对象细节设计的基本原则）
- Design Classes and the Design Class Diagram
- Detailed Design with CRC Cards
- Fundamental Detailed Design Principles

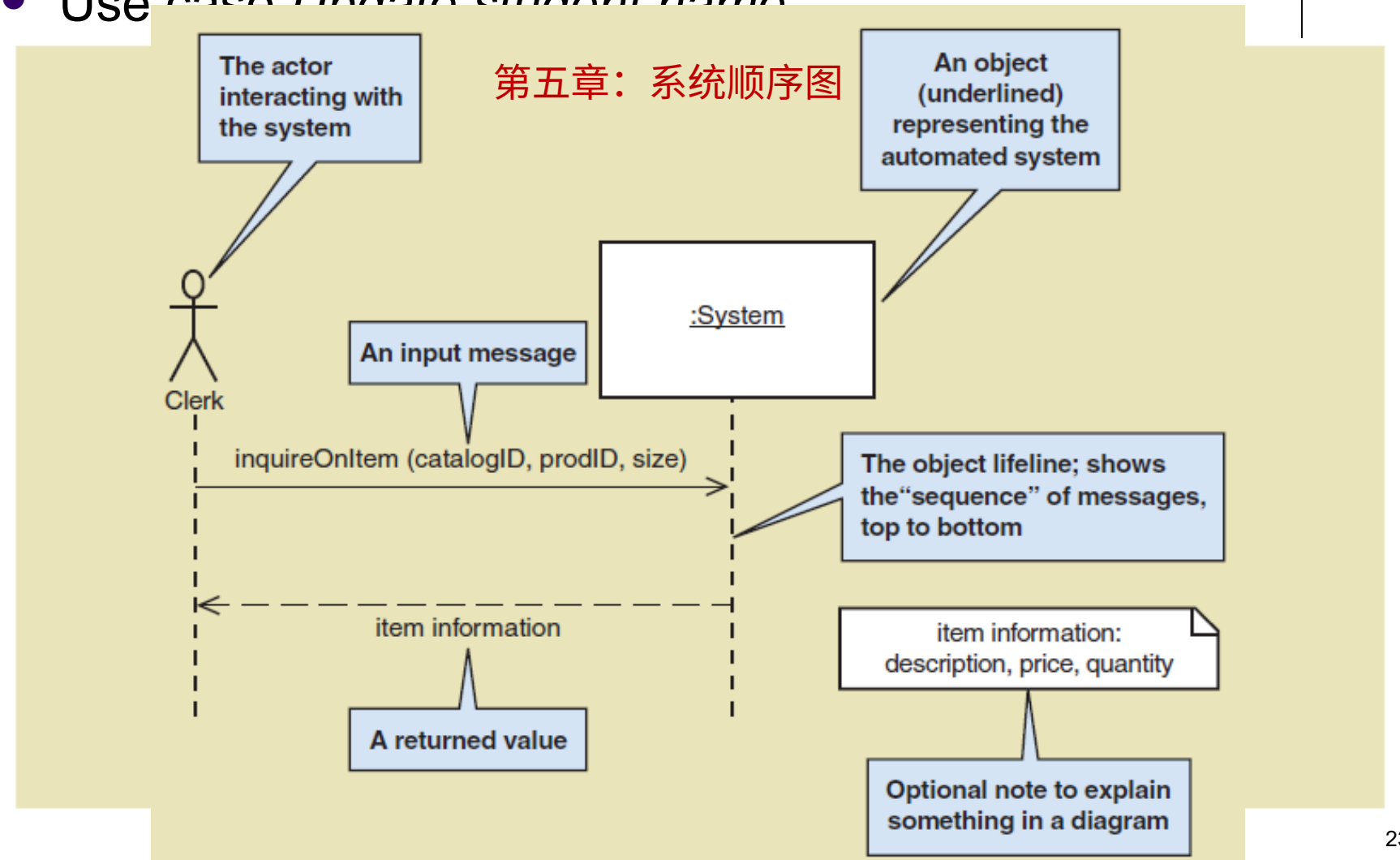
Detailed Design （详细设计）
Use Case Realization （用例实现）



- Design and Implement Use Case by Use Case
  - Sequence Diagram （顺序图）— extended for system sequence diagram （系统顺序图） (Chapter 5) adding a controller and the domain classes
  - Design Class Diagram （设计类图）— extended from the domain model class diagram （域模型类图） (Chapter 4) and updated from sequence diagram
    - Messages to an object become methods of the design class

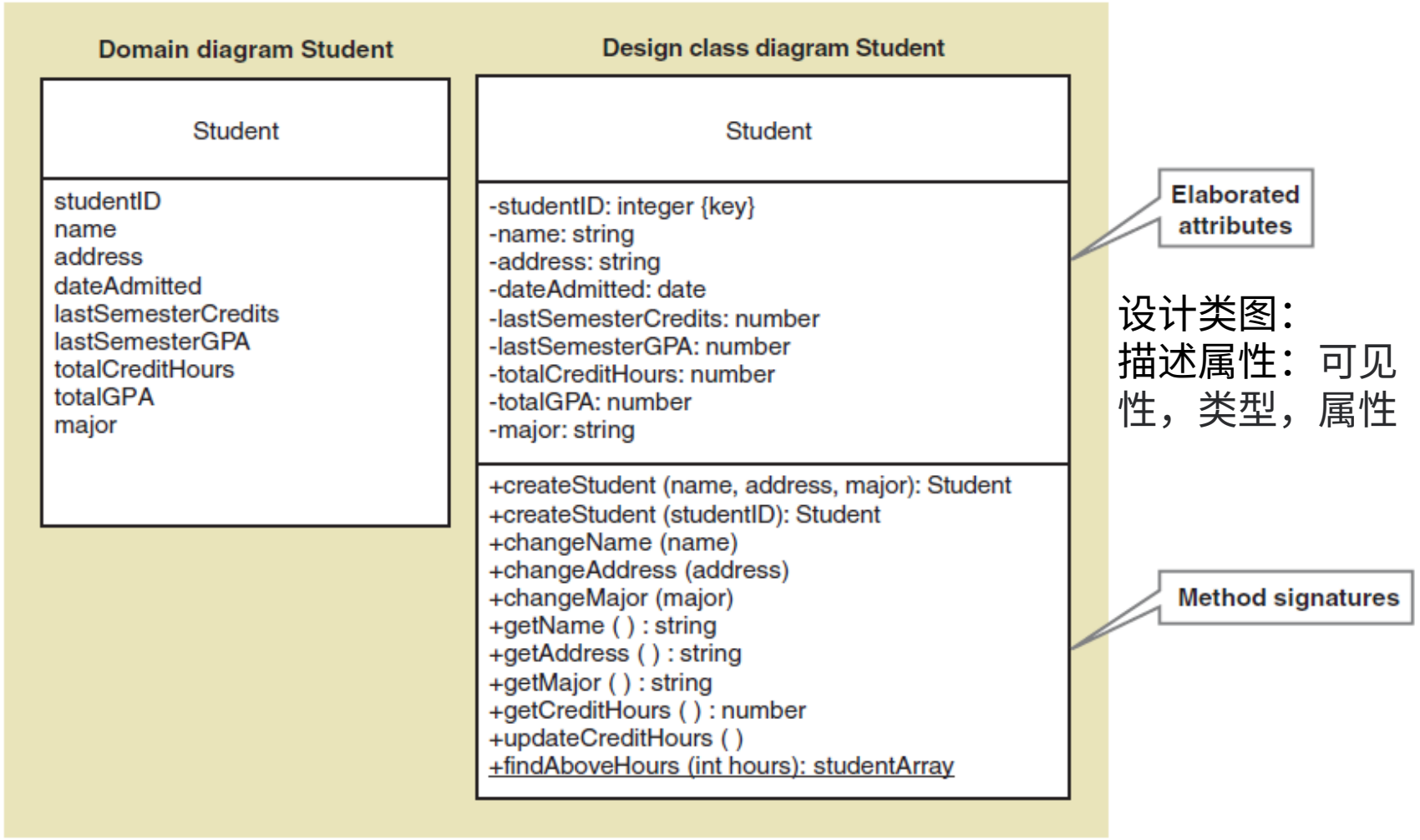
Detailed Design
Sequence Diagram Example

- Use case *Update student name*

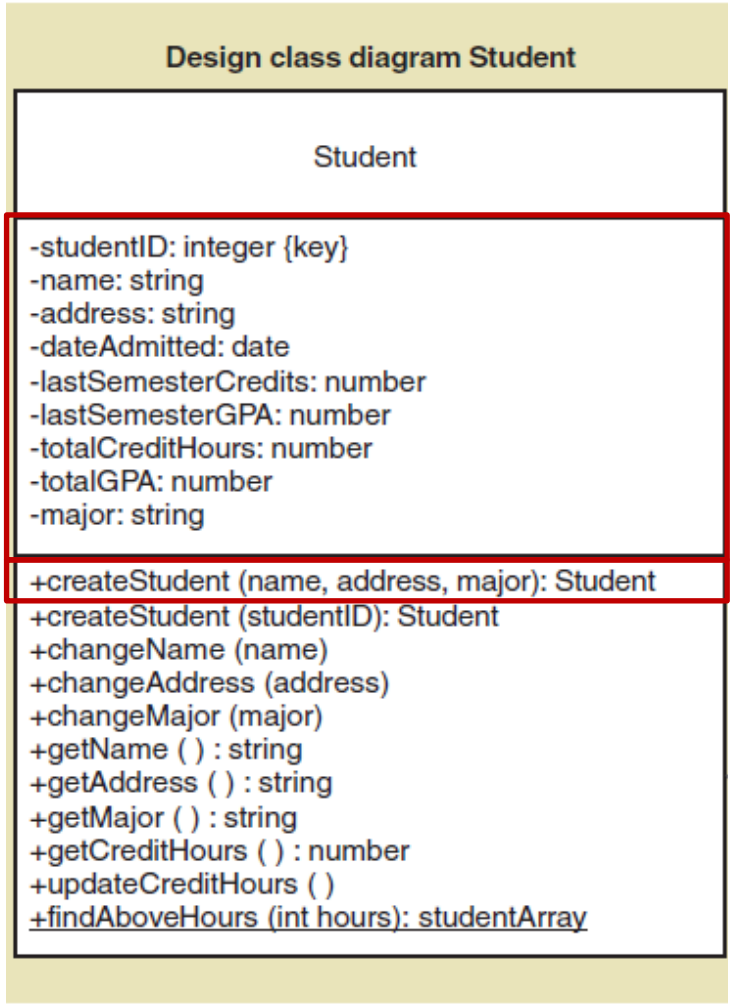


Design Classes in Detailed Design

- Elaborate attributes—visibility, type, properties



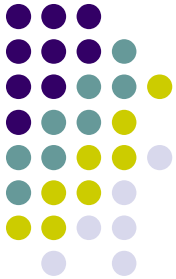
设计类图：
描述属性：可见性，类型，属性




```
public class Student
{
      //attributes
      private int studentID;
      private String firstName;
      private String lastName;
      private String street;
      private String city;
      private String state;
      private String zipCode;
      private Date dateAdmitted;
      private float numberCredits;
      private String lastActiveSemester;
      private float lastActiveSemesterGPA;
      private float gradePointAverage;
      private String major;

      //constructors
      public Student (String inFirstName, String inLastName, String inStreet,
            String inCity, String inState, String inZip, Date inDate)
      {
            firstName = inFirstName;
            lastName = inLastName;
            ...
      }
      public Student (int inStudentID)
      {
            //read database to get values
      }

      //get and set methods
      public String getFullName ( )
      {
            return firstName + " " + lastName;
      }
      public void setFirstName (String inFirstName)
      {
            firstName = inFirstName;
      }
      public float getGPA ( )
      {
            return gradePointAverage;
      }
      //and so on

      //processing methods
      public void updateGPA ( )
      {
            //access course records and update lastActiveSemester and
            //to-date credits and GPA
      }
}
```

OO Detailed Design Steps

Chapters 10 and 11



Design Step	Chapter
1. Develop the first-cut design class diagram showing navigation visibility.	10
2. Determine class responsibilities and class collaborations for each use case using class-responsibility-collaboration (CRC) cards.	10
3. Develop detailed sequence diagrams for each use case. (a) Develop the first-cut sequence diagrams. (b) Develop the multilayer sequence diagrams.	11
4. Update the design class diagram by adding method signatures and navigation information using CRC cards and/or sequence diagrams.	11
5. Partition the solution into packages as appropriate.	11

开发初步设计类图，显示导航可见性

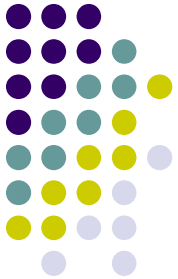
使用 CRC 卡位用例决定类的职责和合作使用

为每个用例开发详细的顺序图：1）开发初步顺序图；2）开发多层顺序图

使用 CRC 卡或顺序图，添加方法特征和导航信息，以此来更新设计类图

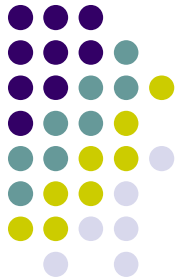
将解决方案划分成适当的包

Chapter 10 Outline



- Object-Oriented Design: Bridging from Analysis to Implementation
- Object-Oriented Architectural Design
- Fundamental Principles of Object-Oriented Detailed Design
- Design Classes and the Design Class Diagram （设计类和设计类图）
- Detailed Design with CRC Cards
- Fundamental Detailed Design Principles

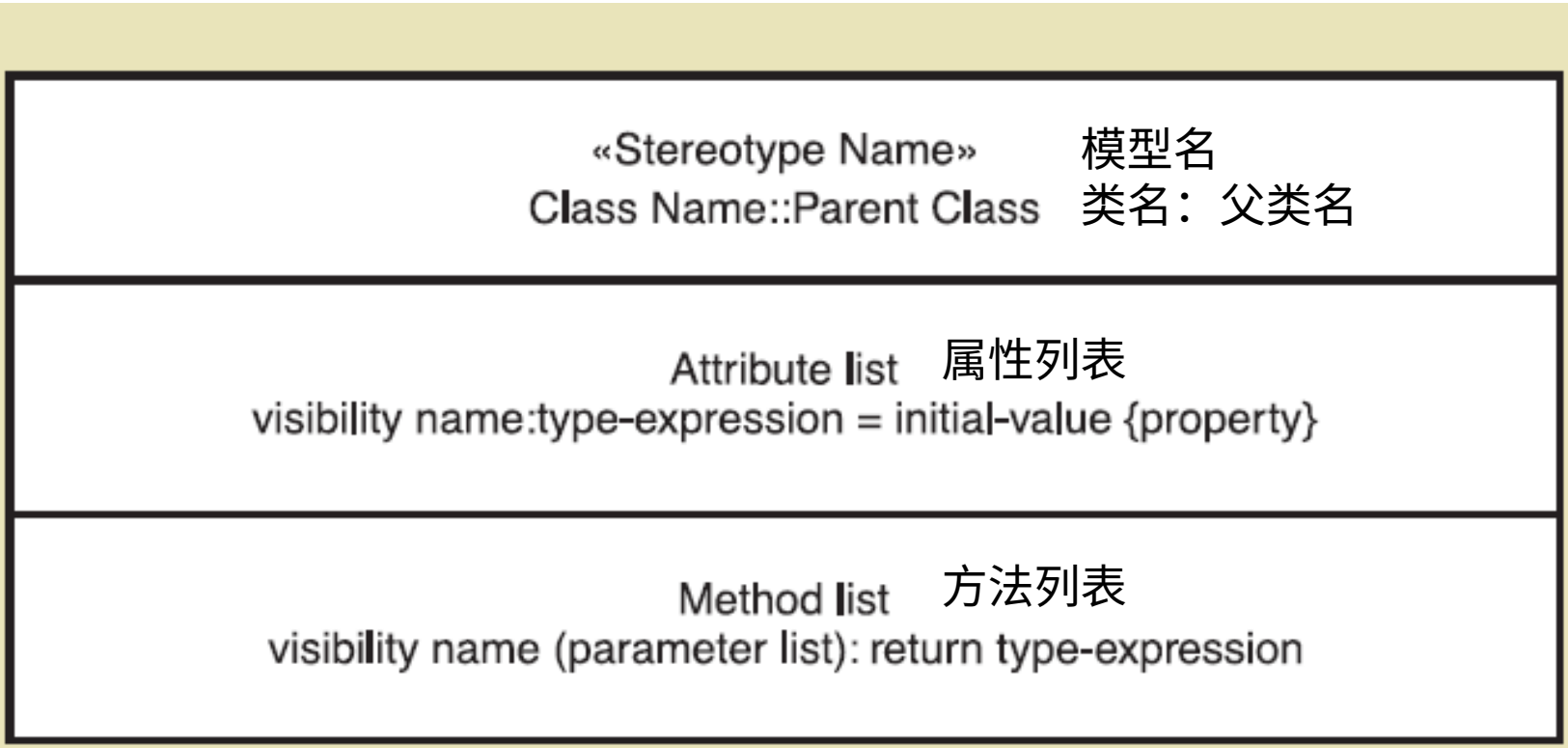
Design Class Diagrams （设计类图）



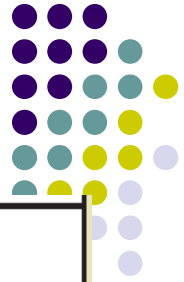
- **Stereotype （构造型）** a way of categorizing a model element by its characteristics, indicated by guillemots (<< >>) （对模型元素以特定的类型分类）
  - **entity class （实体类）** a design identifier for a problem domain class (usually persistent)
    - **persistent class** an class whose objects exist after a system is shut down (data remembered)
  - **boundary class or view class （边界类 / 视图类）** a class that exists on a system's automation boundary, such as an input window form or Web page
  - **control class （控制类）** a class that mediates between boundary classes and entity classes, acting as a switchboard between the view layer and domain layer
  - **data access class （数据访问类）** a class that is used to retrieve data from and send data to a database

Notation for a Design Class

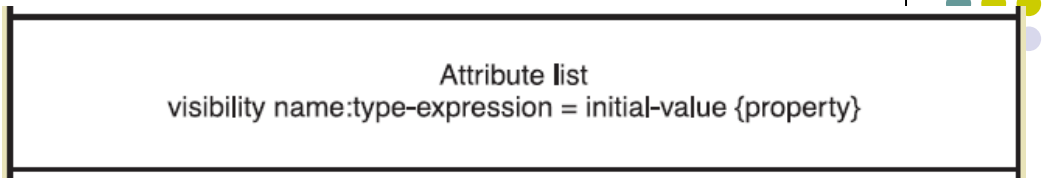
- Syntax for Name, Attributes, and Methods （定义设计类的符号）



Notation for Design Classes



- Attributes （属性）

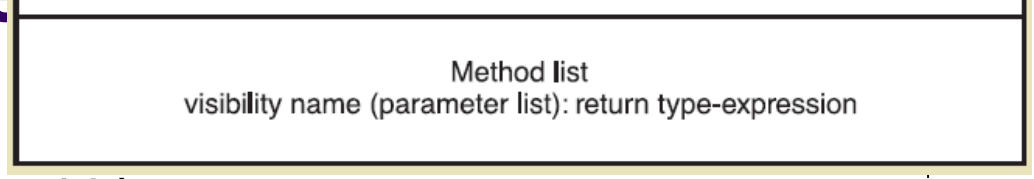


  - Visibility （可见性）— indicates (+ or -) whether an attribute can be accessed ***directly*** by another object. Usually ***private*** (-) not public (+)
  - Attribute name （属性名称）— Lower case camelback notation
  - Type expression （类型表达）— class, string, integer, double, date
  - Initial value （初始值）— if applicable the default value
  - Property （特性）— if applicable, such as {key}
  - Examples:

    -accountNo: String {key}

    -startingJobCode: integer = 01

Notation for Design Classes



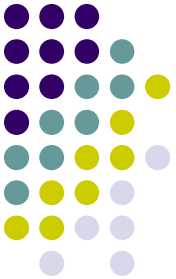
- Methods （方法）
  - Visibility （方法可见性）— indicates (+ or -) whether an method can be invoked by another object. Usually ***public*** (+), can be private if invoked within class like a subroutine
  - Method name （方法名称）— Lower case camelback, verb-noun
  - Parameters （方法参数列表）— variables passed to a method
  - Return type （返回参数类型）— the type of the data returned
  - Examples:

    +setName(fName, lName) : void (void is usually let off)

    +getName(): string (what is returned is a string)

    -checkValidity(date) : int (assuming int is a returned code)

Notation for Design Classes



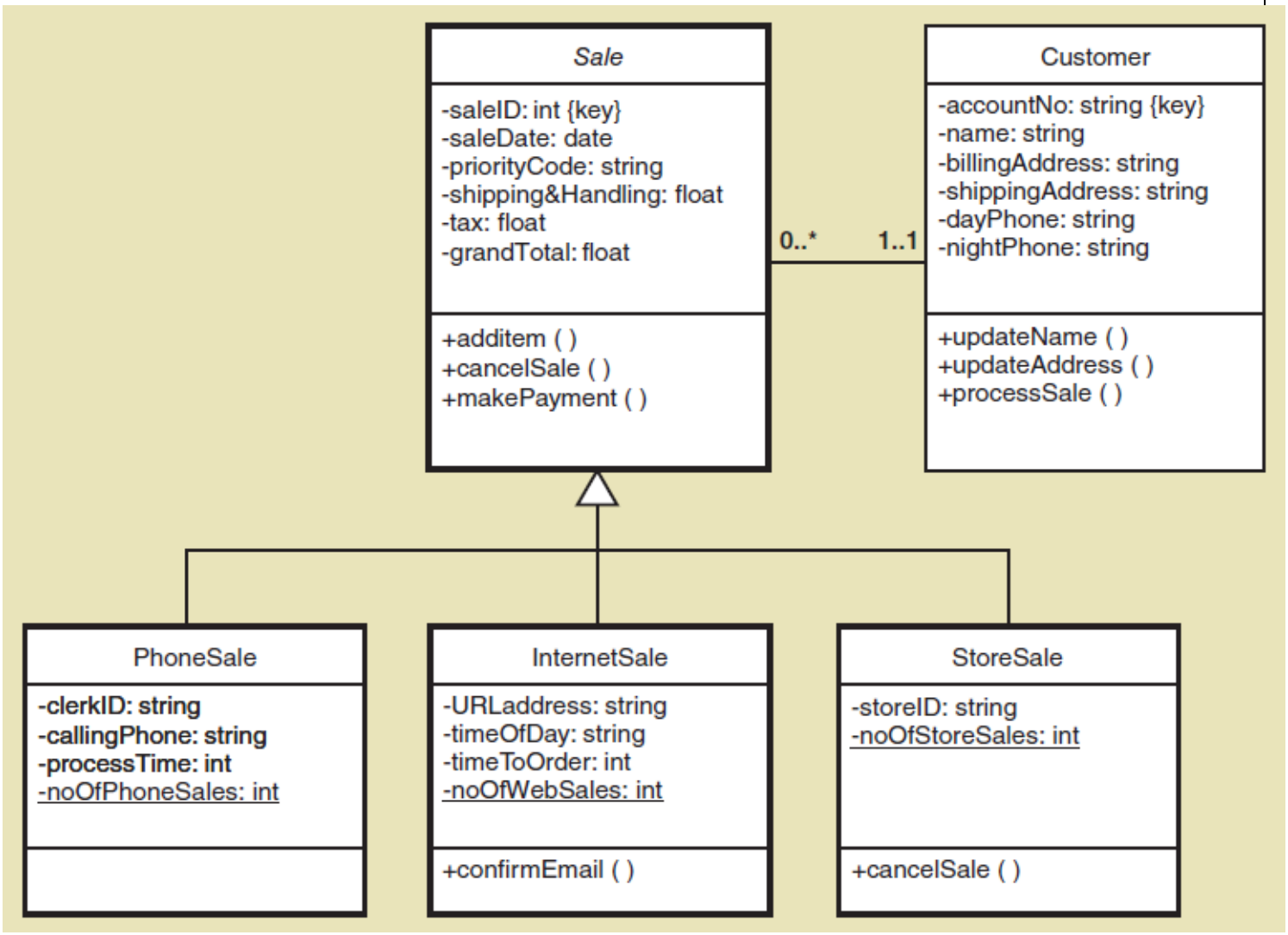
- Class level method （类级方法）— applies to class rather than objects of class (aka static method). （静态方法：应用于类而不是类的对象）
  - +findStudentsAboveHours(hours): Array
  - +getNumberOfCustomers(): Integer
- Class level attribute （类级属性）— applies to the class rather than an object (aka static attribute). （静态属性：应用于类而不是对象）
  - -noOfPhoneSales: int
- 静态：只要定义了类，不必建立类的实例就可以使用

Notation for Design Classes

- Abstract class （抽象类）– class that can’t be instantiated. （不能被实例化的类）
  - Only for inheritance. Name in *Italics*. （仅用于继承）
- Concrete class （具体类）— class that can be instantiated. （能被实例化的类）

Notation for Design Classes

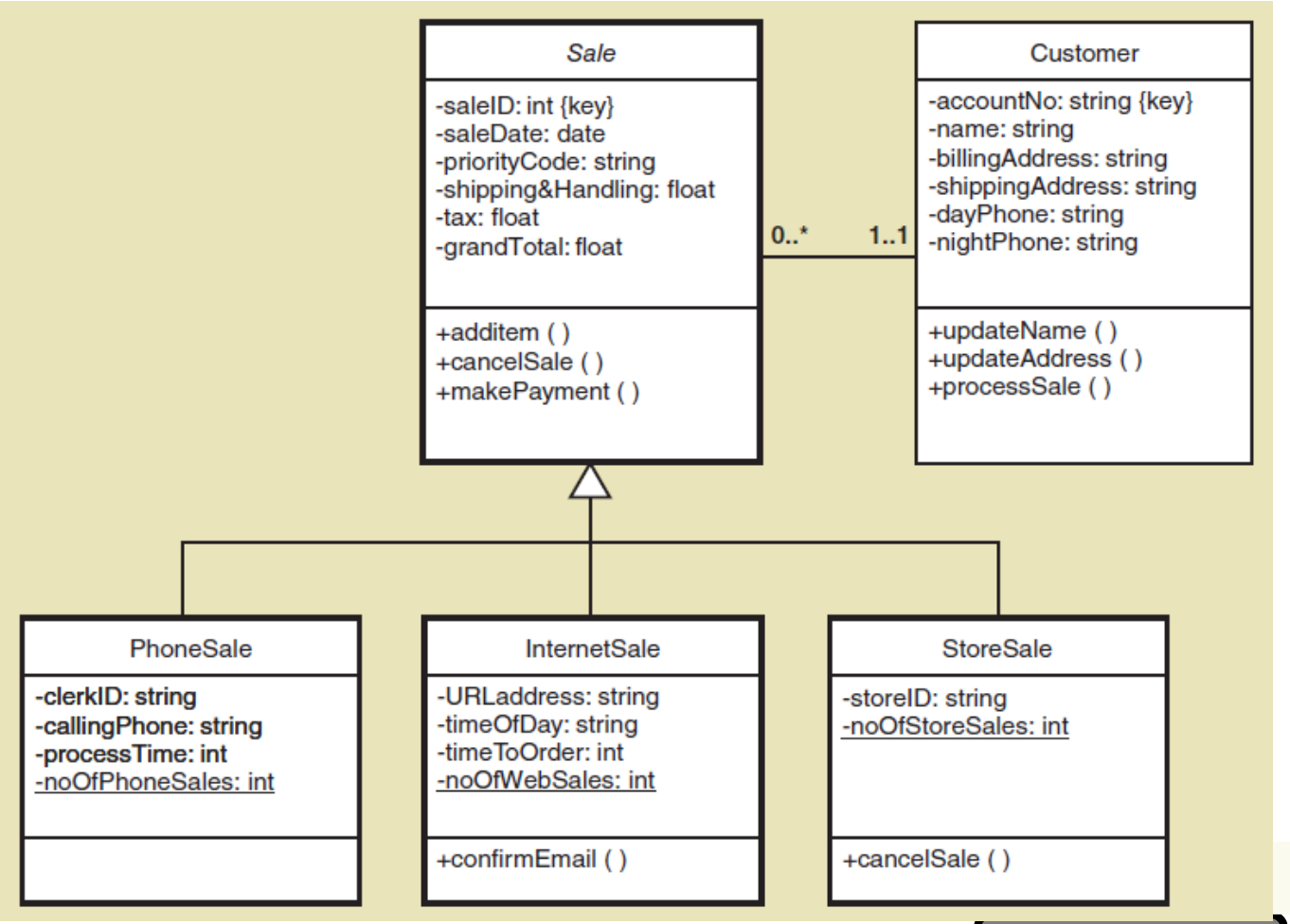
method arguments and return types not shown



主观题 10 分

设置

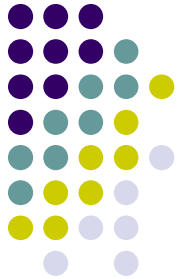
Write Java code frame according to the following class diagram.



Systems Analysis and Design in a Changing World,

作答

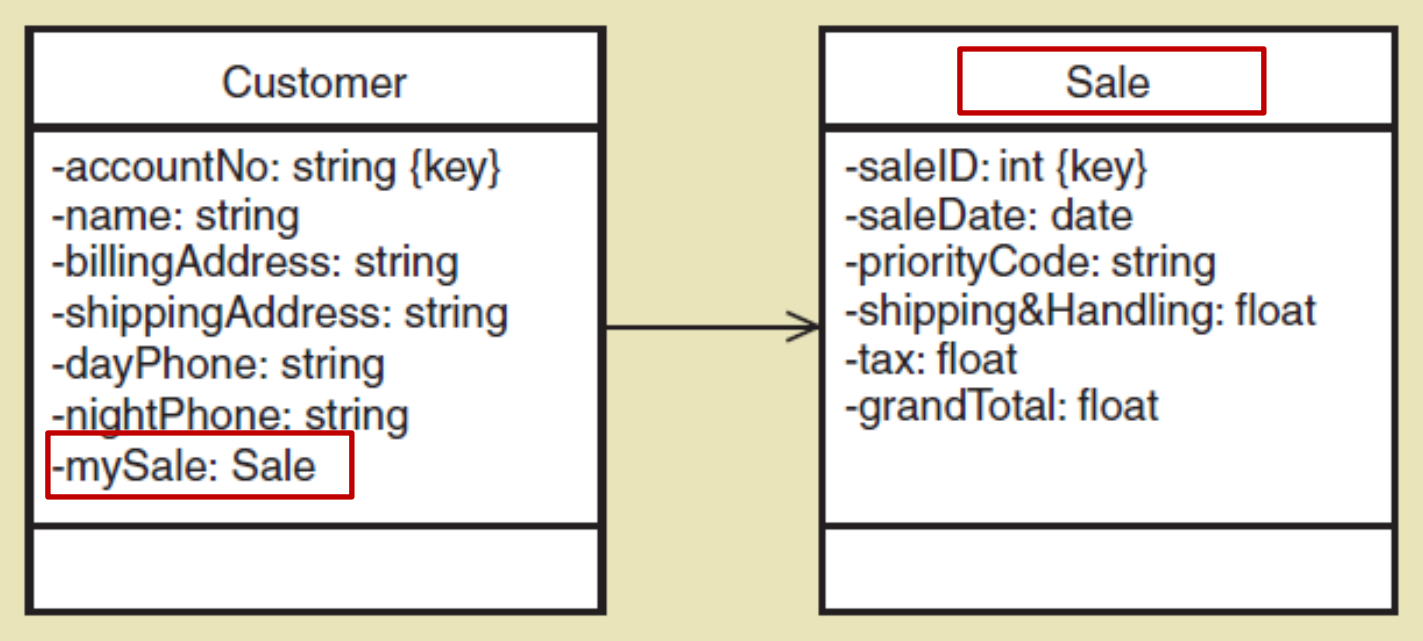
Notation for Design Classes



- Navigation Visibility （导航可见性）
  - The ability of one object to view and interact with another object（一个对象查看另一个对象并与之交互的能力）
  - Accomplished by adding an object reference variable to a class.（通过向类添加对象引用变量来实现）
  - Shown as an arrow head on the association line—customer can find and interact with sale because it has mySale reference variable （在关联线上显示为箭头，客户可以找到 sale 并与之交互，因为它有 mysql 引用变量）

Notation for Design Classes

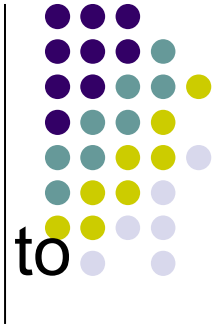
- Navigation Visibility （导航可见性）



Navigation Visibility Guidelines

- One-to-many associations （一对多关联） that indicate a superior/subordinate relationship are usually navigated from the superior to the subordinate （从上级导航到下级）
- Mandatory associations （强制关联）, in which objects in one class can't exist without objects of another class, are usually navigated from the more independent class to the dependent （一个类中的对象不能没有另一个类的对象而存在，通常从更独立的类导航到依赖的类）
- When an object needs information from another object, a navigation arrow might be required （当一个对象需要来自另一个对象的信息时，可能需要一个导航箭头）
- Navigation arrows may be bidirectional. （导航箭头可以是双向的）

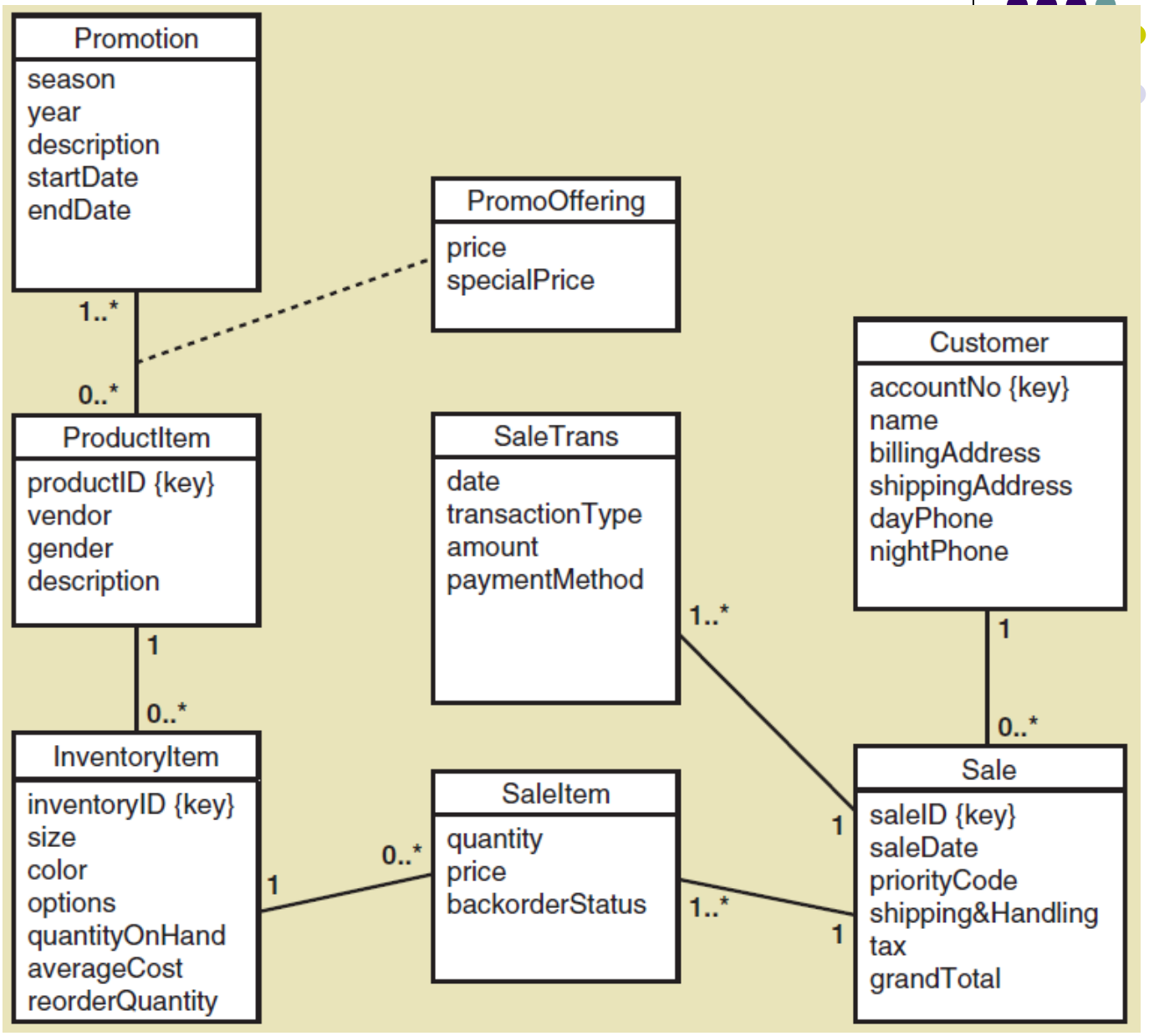
First Cut Design Class Diagram



- Proceed use case （用例） by use case, adding to the diagram
- Pick the domain classes （域类） that are involved in the use case (see preconditions and post conditions for ideas)
- Add a controller class （控制器类） to be in charge of the use case
- Determine the initial navigation visibility （导航可见性） requirements using the guidelines and add to diagram
- Elaborate the attributes of each class with visibility and type （用可见性和类型详细说明每个类的属性）

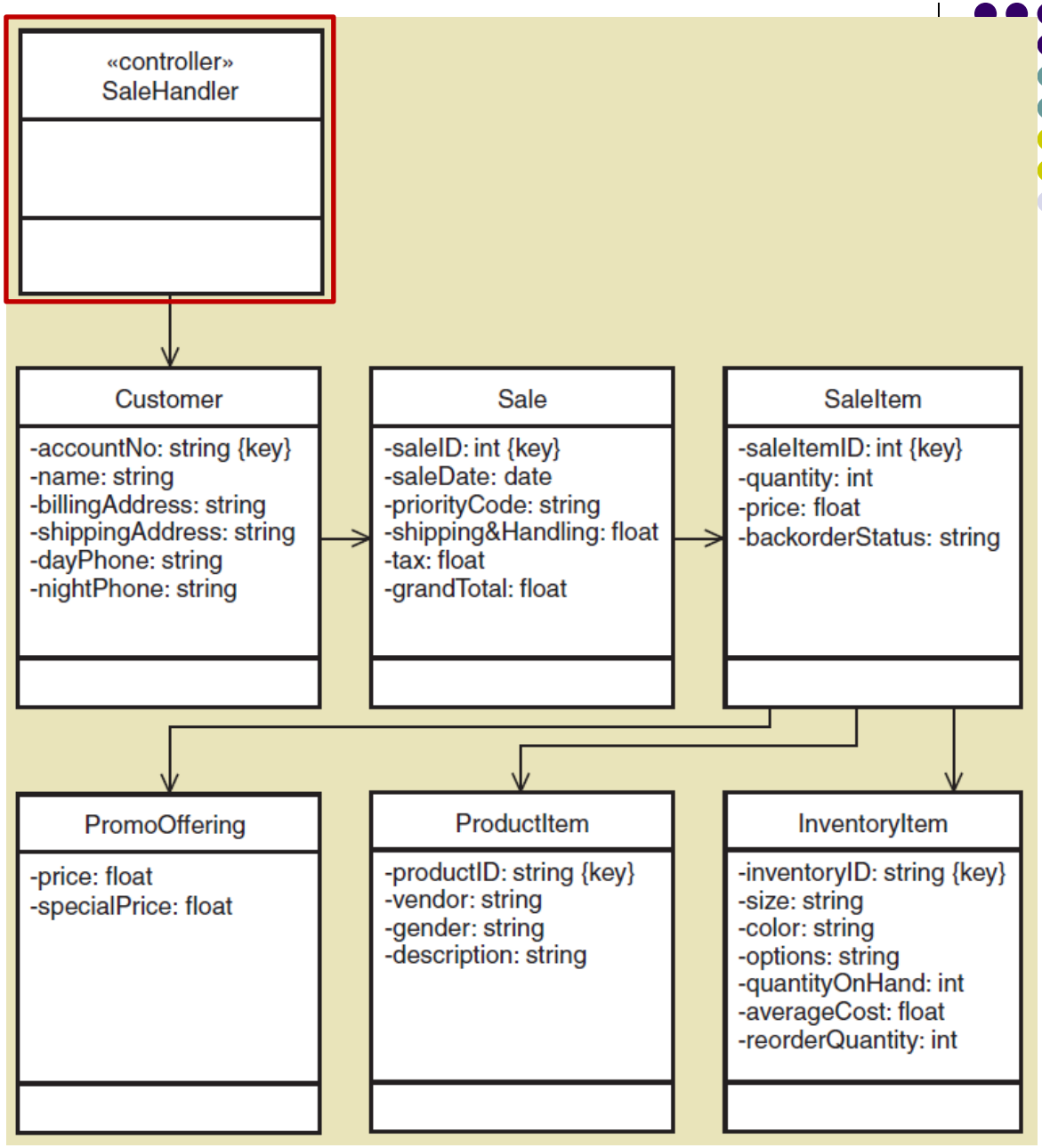
Start with Domain Class Diagram （域模型类图：第四章）

RMO Sales Subsystem



Create First Cut Design Class Diagram

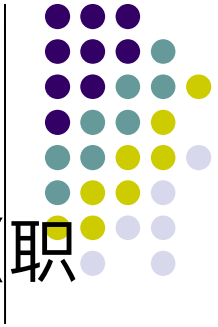
Use Case *Create phone sale* with controller added



Chapter 10 Outline

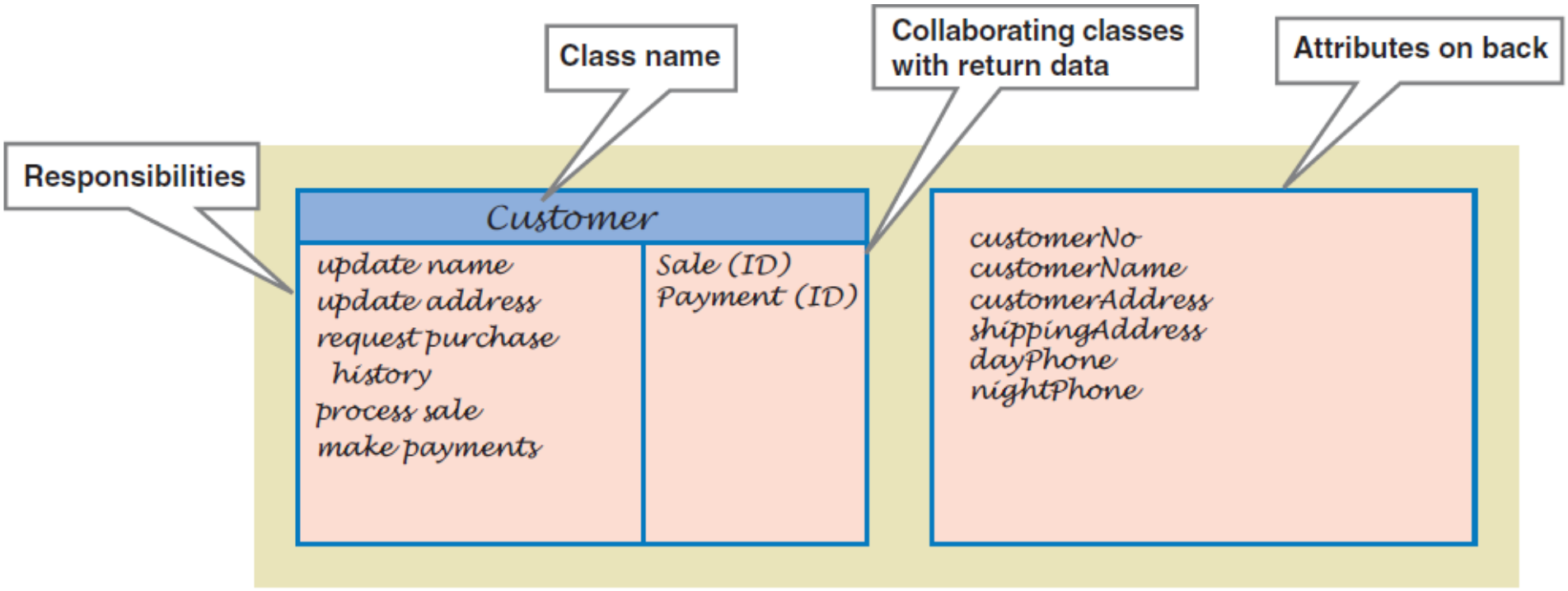
- Object-Oriented Design: Bridging from Analysis to Implementation
- Object-Oriented Architectural Design
- Fundamental Principles of Object-Oriented Detailed Design
- Design Classes and the Design Class Diagram
- Detailed Design with CRC Cards （详细设计与 CRC 卡）
- Fundamental Detailed Design Principles

Designing With CRC Cards



- CRC Cards—Classes （类）, Responsibilities （职责）, Collaboration （协作） Cards
- OO design is about assigning Responsibilities to Classes for how they Collaborate to accomplish a use case （OO 设计是关于为类分配职责，以便它们如何协作来完成用例）
- Usually a manual process done in a brainstorming session
  - One card per class
  - Front has responsibilities and collaborations （正面：责任和协作）
  - Back has attributes needed （反面：属性）

Example of CRC Card



CRC Cards Procedure

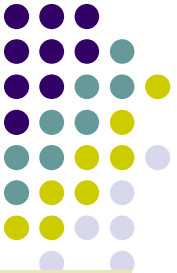
- Because the process is to design, or realize, a single use case, start with a set of unused CRC cards. Add a controller class (Controller design pattern) （从一组未使用的 CRC 卡开始，添加一个控制器类）.
- Identify a problem domain class that has primary responsibility for this use case that will receive the first message from the use case controller. （确定一个问题域类）
- Use the first cut design class diagram to identify other classes that must collaborate with the primary object class to complete the use case. （识别必须与主要对象类协作以完成用例的其他类）

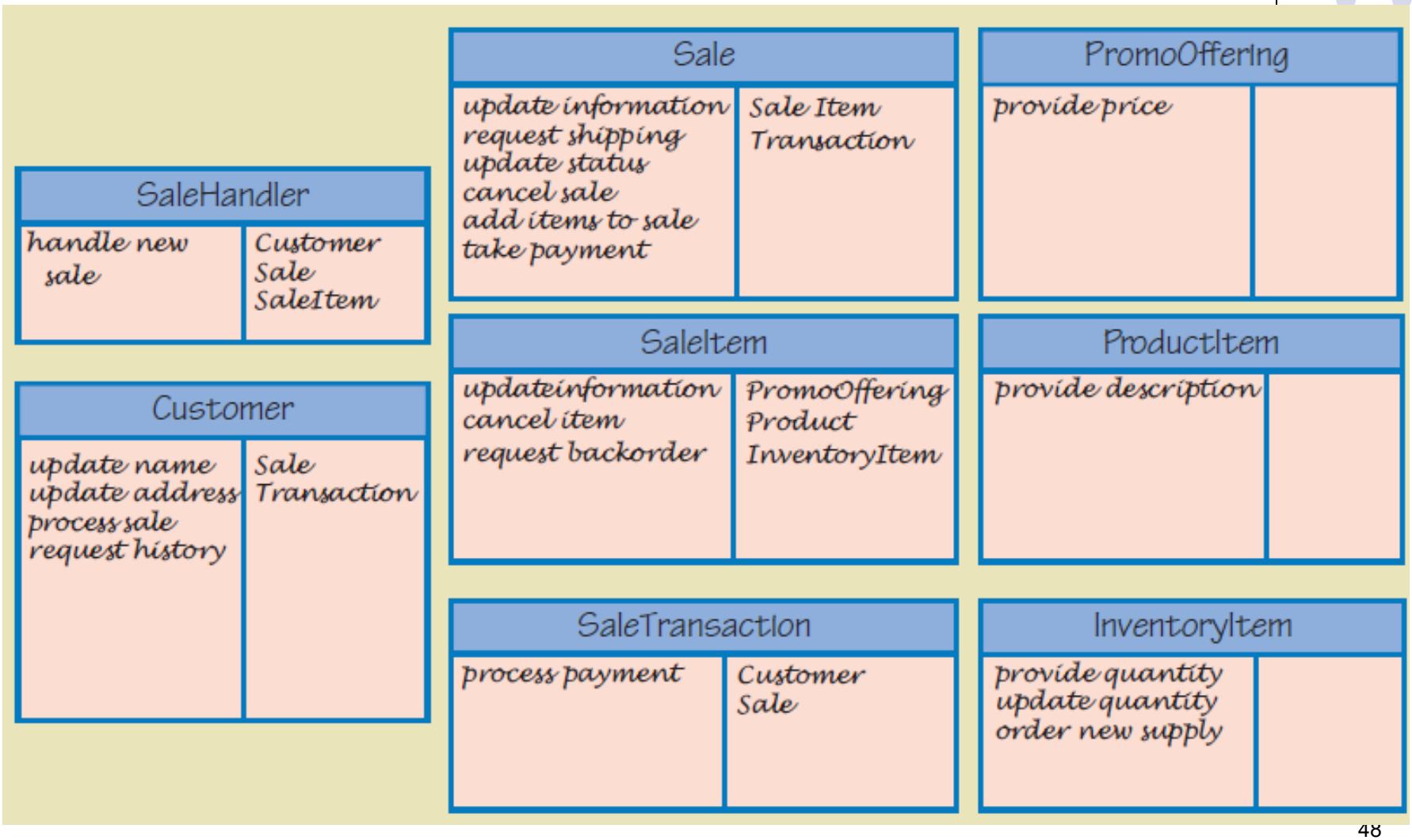
CRC Cards Procedure (continued)

- Start with the class that gets the first message from the controller. Name the responsibility （命名职责） and write it on card.
- Now ask what this first class needs to carry out the responsibility. Assign other classes responsibilities to satisfy each need （分配其他类职责以满足每个需求。把责任写在卡片上）. Write responsibilities on those cards.
- Add collaborators to cards showing which collaborate with which. Add attributes to back when data is used
- Eventually, user interface classes （用户交互类） or even data access classes （数据访问类） can be added

CRC Cards Results

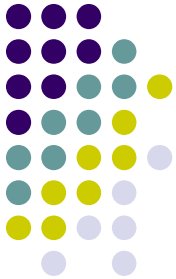
Several Use Cases

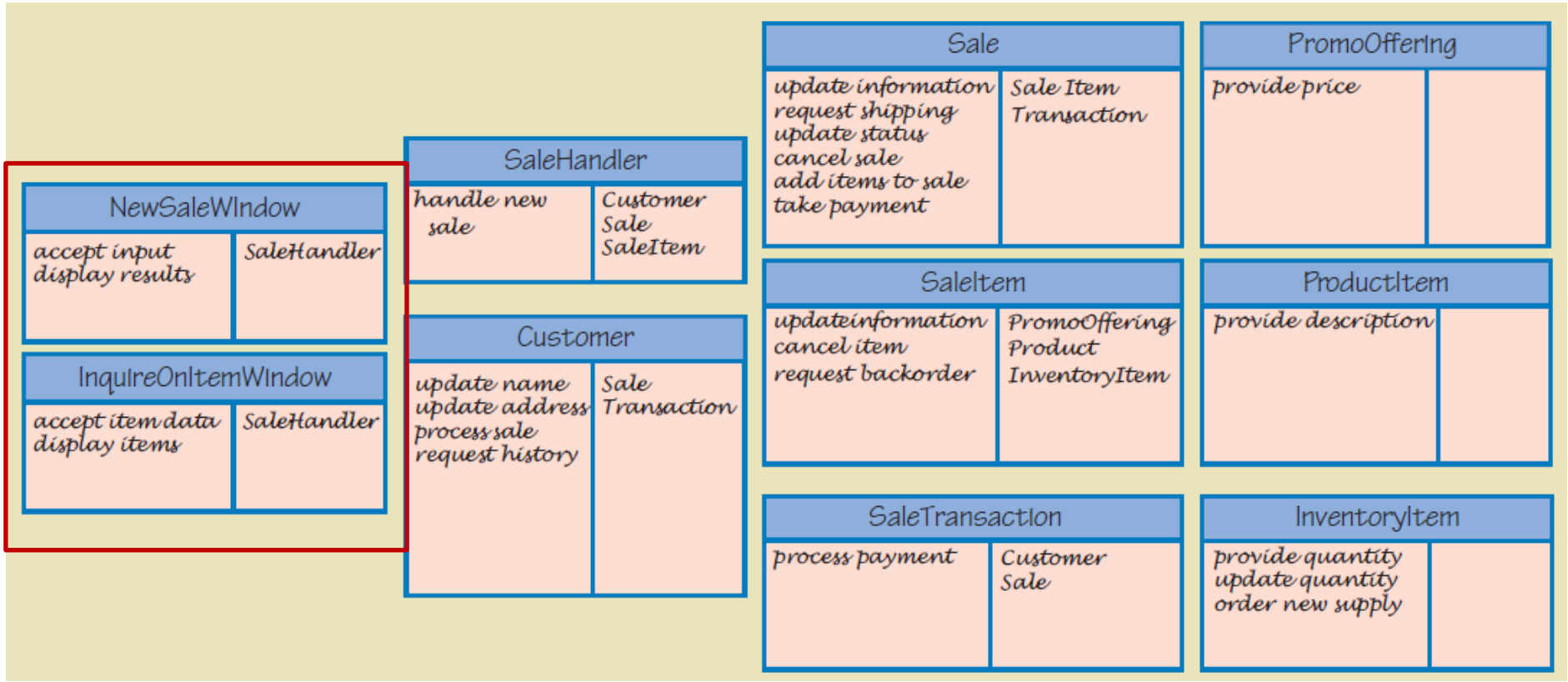




CRC Cards Results

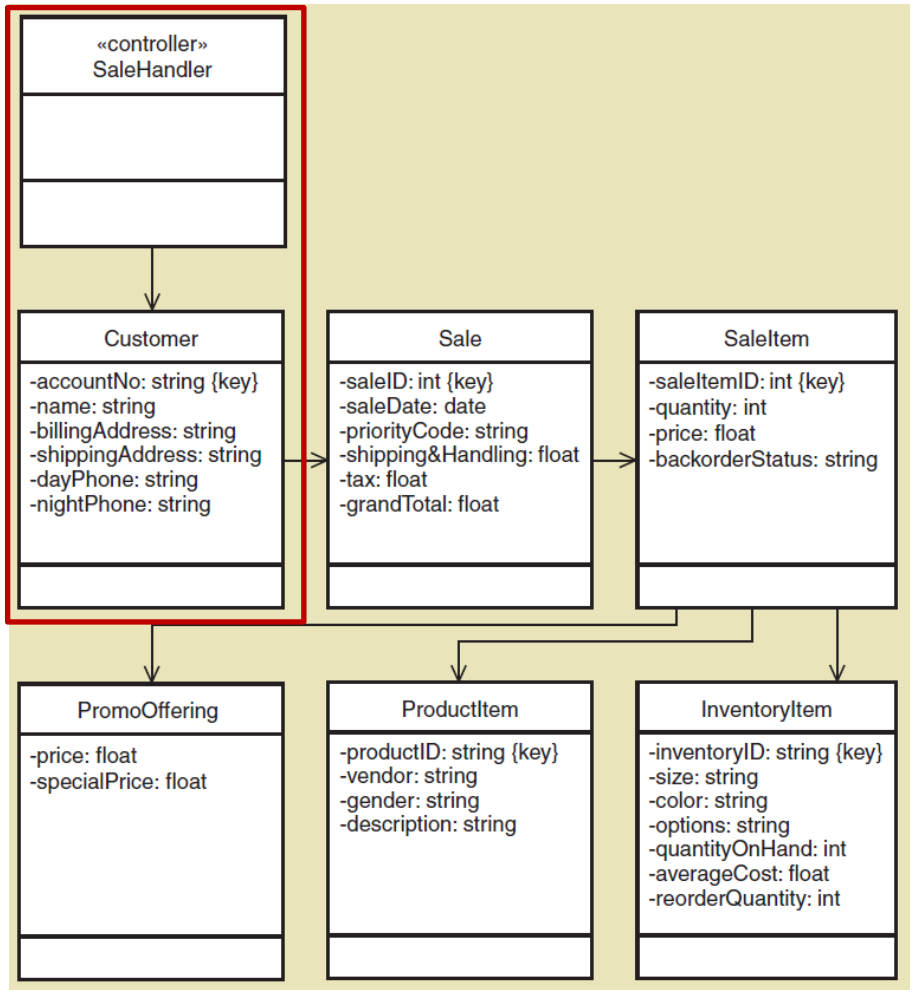
Adding In User Interface Layer

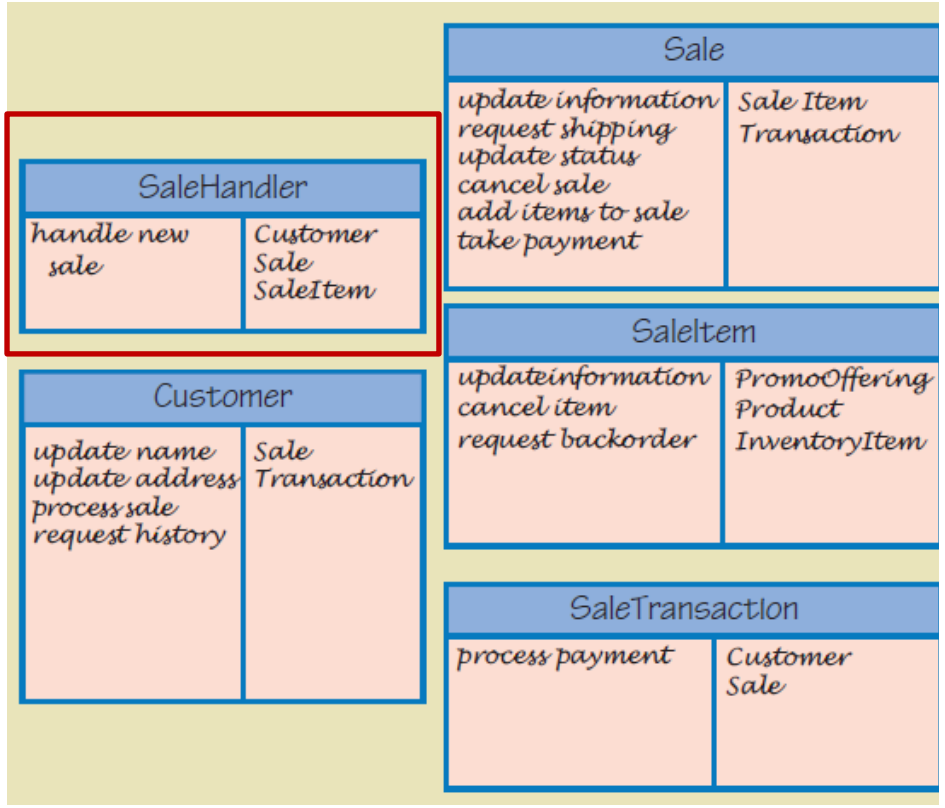




CRC Cards Results

Several Use Cases



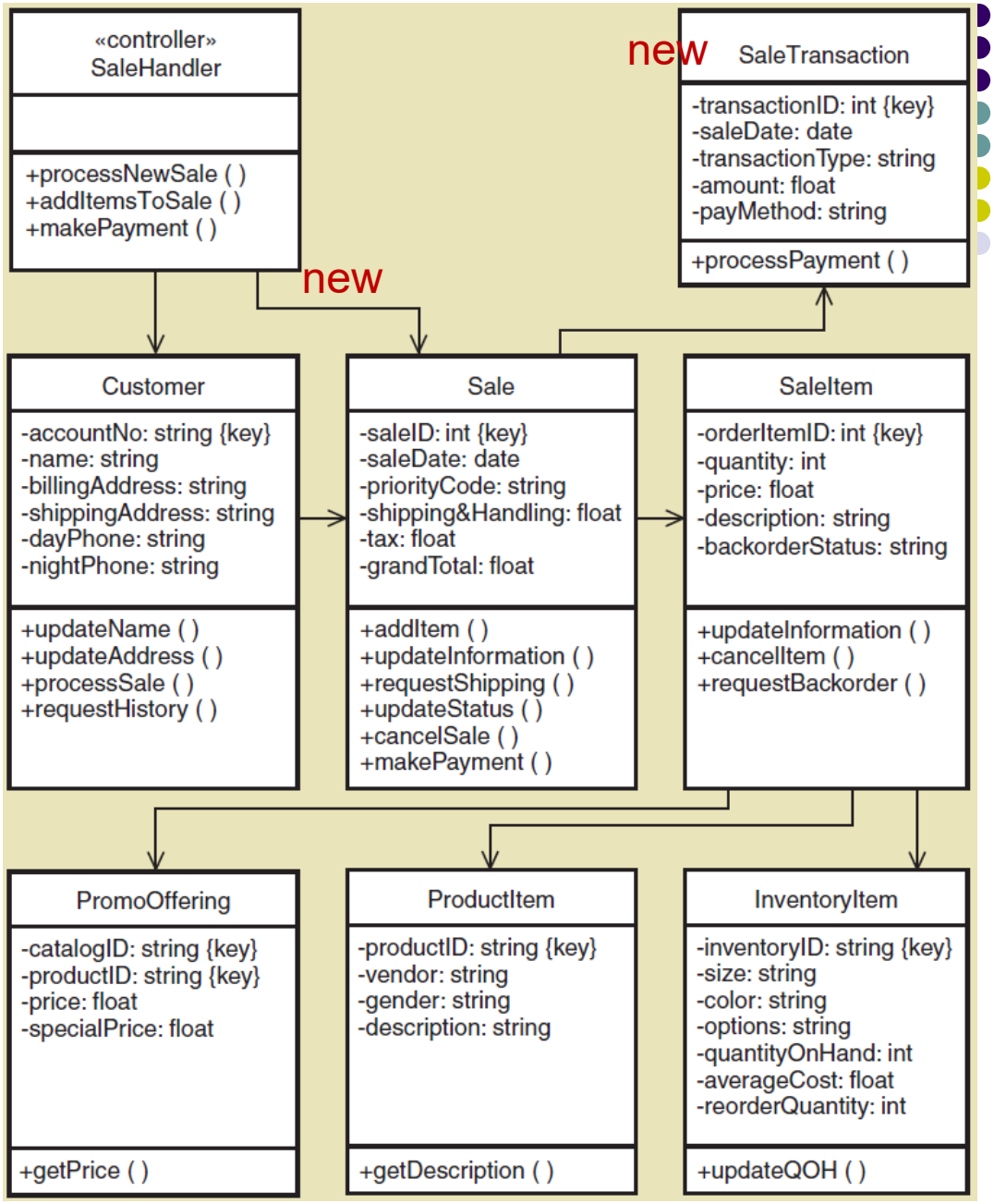


CRC Cards

Update design class diagram based on CRC results

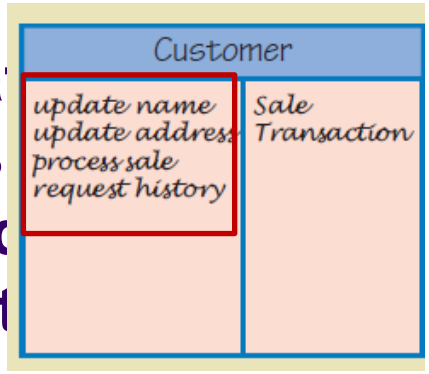
Responsibilities become methods

Arguments and return types not yet added



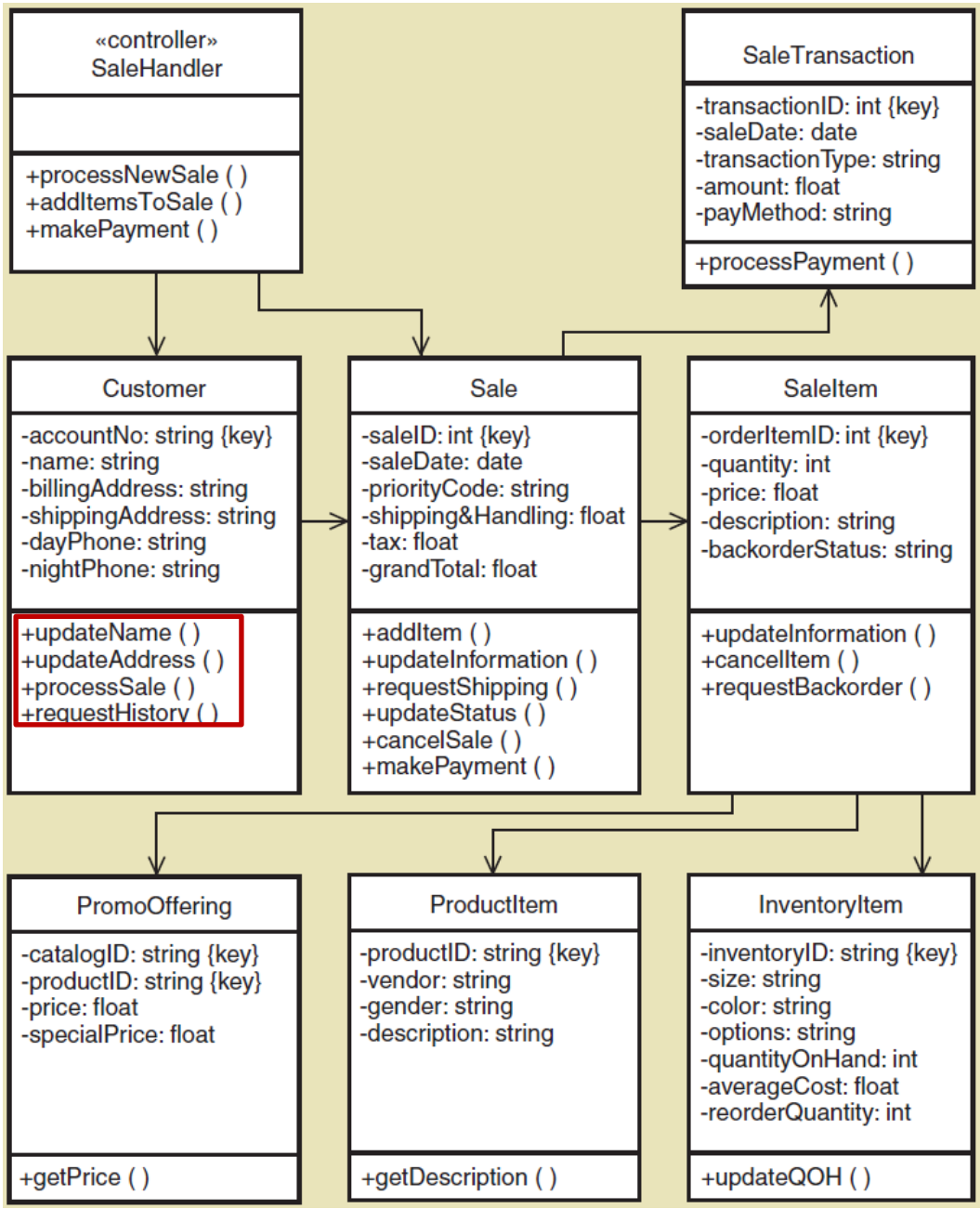
CRC Cards

Upda
class
based
result



Responsibilities become methods

Arguments and return types not yet added

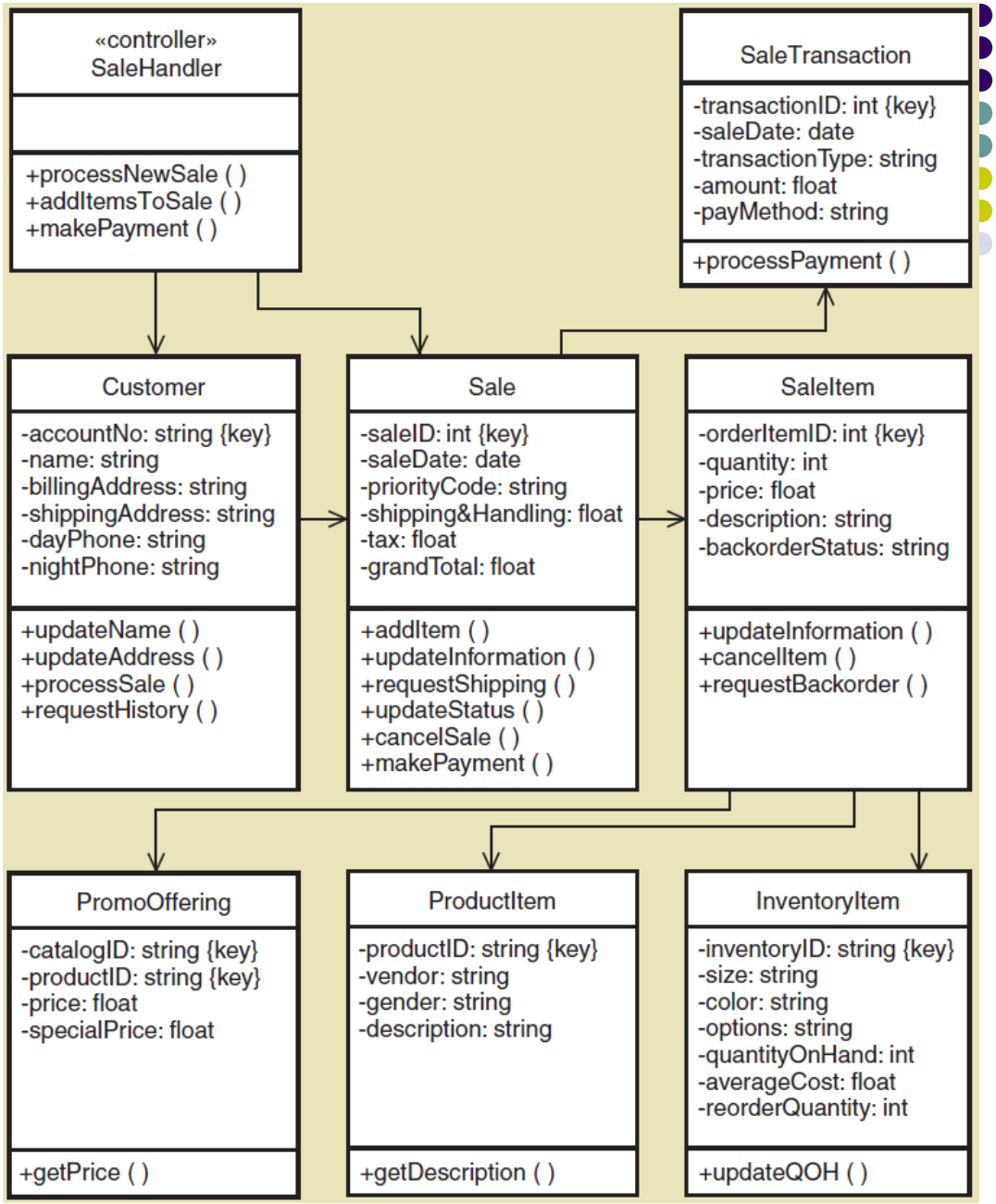


CRC Cards

Update design class diagram based on CRC results

Responsibilities become methods

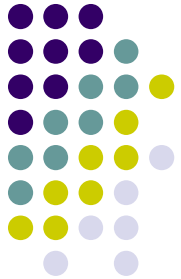
Arguments and return types not yet added



Chapter 10 Outline

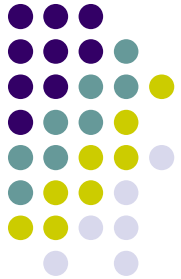
- Object-Oriented Design: Bridging from Analysis to Implementation
- Object-Oriented Architectural Design
- Fundamental Principles of Object-Oriented Detailed Design
- Design Classes and the Design Class Diagram
- Detailed Design with CRC Cards
- Fundamental Detailed Design Principles

Fundamental Design Principles



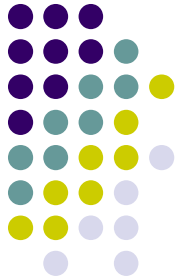
- Coupling （耦合性）
  - A quantitative measure of how closely related classes are linked (tightly or loosely coupled) （对相关类的链接紧密程度（紧密耦合或松散耦合）的定量度量）
  - Two classes are tightly coupled of there are lots of associations with another class （两个类是紧密耦合的）
  - Two classes are tightly coupled if there are lots of messages to another class （如果有很多传递给另一个类的消息，则两个类是紧密耦合的）（互相通信的对象必须有导航可见性，所以他们就是耦合的）
  - It is best to have classes that are **loosely coupled （松耦合）（强耦合时，一个类的变化会波及整个系统）**

Fundamental Design Principles



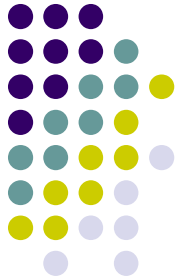
- Cohesion （内聚）
  - A quantitative measure of the focus or unity of purpose within a single class (high or low cohesiveness) （内聚是对一个类的功能一致性的定性度量）
  - One class has high cohesiveness if all of its responsibilities are consistent and make sense for purpose of the class （如果一个类的所有职责都是一致的，并且对类的目的有意义，则该类具有高内聚性）
  - One class has low cohesiveness if its responsibilities are broad or makeshift （如果一个类的职责很宽泛或者是临时的，那么它的内聚性就很低）
  - It is best to have classes that are **highly cohesive （高内聚性）**

Fundamental Design Principles



- Protection from Variations （变量保护）
  - A design principle that states parts of a system unlikely to change are separated (protected) from those that will surely change （将系统中不太可能改变的部分与那些肯定会改变的部分分开（保护）的设计原则）
  - Separate user interface （接口逻辑） forms and pages that are likely to change from application logic （业务逻辑分离） （用户接口）
  - Put database connection and SQL logic that is likely to change in a separate classes from application logic （数据库连接和业务逻辑分离）
  - Use adaptor classes that are likely to change when interfacing with other systems （使用在与其他系统接口时可能会更改的适配器类）

Fundamental Design Principles

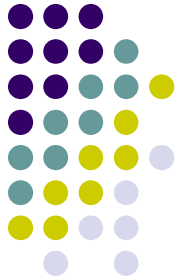


- Indirection （间接）
  - A design principle that states an intermediate class is placed between two classes to decouple them but still link them （通过放置一个充当链接的中间件来分离两个类或其他系统组件的原则）
  - A controller class （控制器类） between UI classes and problem domain classes is an example
  - Supports low coupling （支持低耦合）
  - Indirection is used to support security by directing messages to an intermediate class as in a firewall （通过将消息定向到防火墙中的中间类来支持安全性）

Fundamental Design Principles

- Object Responsibility （对象职责）
  - A design principle that states objects （对象） are responsible for carrying out system processing
  - Responsibilities include “knowing” and “doing” （职责分为“认识”和“行动”）
  - Objects know about other objects (associations) and they know about their attribute values. （对象知道其他关联）
  - Objects know how to carry out methods, do what they are asked to do. （对象知道如何执行方法）
  - 一个对象应该具有对向它提供信息的对象的导航可见性

Summary

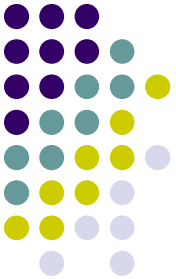


- This chapter focused on designing software that solves business problems by bridging the gap between analysis and implementation. （弥合分析和实现）
- Architectural design is the first step, and the UML component diagram is used to model the main components and application program interfaces (API) （架构设计是第一步，UML 组件图用于对主要组件和应用程序编程接口 (API) 建模）
- Detail design of software proceeds use case by use case, sometimes called “use case driven” and the design of each use case is called use case realization. （软件的详细设计是一个用例接一个用例进行的，有时称为“用例驱动”，每个用例的设计称为用例实现）
- Detailed design models used are design class diagrams and sequence diagrams. （详细设计模型使用的是设计类图和顺序图）

Summary

- The design class diagram is developed is two steps: The first cut diagram is based on the domain model class diagram （设计类图）, but then it is expanded as responsibilities are assigned and sequence diagrams （顺序图） are developed.
- Design class diagrams include additional notation because design classes are now software classes, not just work concepts. （设计类图包含了额外的标记，因为设计类现在是软件类，而不仅仅是工作概念）
- Key issues are attribute elaboration and adding methods. Method signatures include visibility, method name, arguments, and return types. （关键问题是属性的细化和添加方法。方法签名包括可见性、方法名、参数和返回类型）
- Other key terms are abstract vs. concrete classes, navigation visibility, and class level attributes and methods. （其他关键问题包括抽象类与具体类、导航可见性以及类的属性和方法）

Summary (continued)



- CRC Cards technique can be used to design how the classes collaborate to complete each use case. CRC stands for Classes, Responsibilities, and Collaborations. （CRC 卡技术可用于设计类如何协作以完成每个用例。CRC 代表类、责任和协作）
- Sometimes the CRC cards approach is used for the initial design of a use case that is further developed using sequence diagrams (as in the next chapter). （有时 CRC 卡方法用于用例的初始设计，然后使用顺序图进一步开发）
- Decisions about design options are guided by some fundamental design principles. Some of these are coupling, cohesion, protection from variations, indirection, and object responsibility. （关于设计选项的决策是由一些基本的设计原则指导的。其中一些是耦合、内聚、变量保护、间接和对象责任）